

Foundations of Artificial Intelligence

Spring, 2026

Design and Comparative Evaluation of Different AI Algorithms for Mancala

Group 8
Shuyang Huang
Elliott Armstrong
Matthew Kolker

Document Control

Work carried out by:

Full Name	Exhaustive list of Tasks
Shuyang Huang	Wrote the related work section; finished the Minimax and Alpha-beta pruning agents; wrote the importance and impact section; wrote the data collection and preprocessing section; adding visualization contents in the results sections
Elliott Armstrong	Wrote introduction; wrote problem statement; inserted equation formatting; began review of report content and formatting; implemented edits and corrections; provided visuals; organized report content; populated and organized presentation;
Matthew Kolker	Developed game engine, finished the MCTS agent, ran simulations, drafted the methodology, results and interpretations, and discussion and results sections; drafted appendix

Revision Sheet

Date	Revision Description
5 March 2026	Initial draft to include introduction section;
19 March 2026	Initial draft to the related work section
22 March 2026	MK: added methodology/heuristics and challenges (will need to be updated when codebase and analysis is complete)
31 March 2026	Began formatting and clerical revisions
12 April 2026	Implemented first batch of edits and corrections (294 instances)
17 April 2026	Implemented second batch of edits and corrections (563 instances)
23 April 2026	Final comments added, formatting finished, last check initiated

TABLE OF CONTENTS

1 Introduction....4
2 Problem Statement....5
3 Challenges....5
3.1 Related Works....7
3.2 Importance and Impacts....9
4 Data Collection and Preprocessing....9
4.1 Data Collection....9
4.2 Preprocessing....10
5 Methodology....11
5.1 Heuristic Evaluation Function Design....11
5.2 Game Environment....12
5.3 Agent Descriptions....12
5.3.1 Random Agent....12
5.3.2 Greedy Agent....12
5.3.3 Minimax Agent....12
5.3.4 Alpha-Beta Agent....13
5.3.5 Monte Carlo Tree Search Agent....13
5.4 Experimental Protocol....13
5.5 Software Architecture....14
6 Results and Interpretation....14
6.1 Agent Hierarchy....15
6.2 Alpha-Beta Pruning vs. Minimax....16
6.3 Effect of Search Depth....17
6.4 MCTS Simulation Budget and Scaling....17
6.6 Evaluation Function....18
6.7 Extra Turns as a Tactical Mechanism....18
6.8 First-Move Advantage....19
7 Discussion of Results....19
7.1 Summary of Findings....19
7.2 The Intelligence and First-Mover Relationship....20
7.3 Relationship to Prior Work....21
7.4 Practical Implications....21
7.5 Limitations....21
7.6 Future Work....21
8 Your Feedback....22
9 References....23

Appendix **24**
Appendix A: Complete Agent Summary and Elo Ratings 25
Elo Computation Methodology 25
Sensitivity Analysis 25
Appendix B: MCTS Scaling Analysis 28
Appendix C: Exploration Constant Sensitivity 29
Appendix D: Evaluation Function Comparison 30
Appendix E: Extra Turn Analysis 31
Table E-1 31
Table E-2 31
Table E-3 32
Table E-4 32
Appendix F: First-Mover Advantage 34
Table F-1 34
Table F-2 34
Table F-3 34
Statistical Tests 35
Appendix G: Pruning Efficiency 37
Appendix H: Depth Scaling 38
Appendix I: Cost-Performance Analysis 39
Appendix J: Predictive Modeling 41
MCTS Logistic Scaling Model 41
Table J-2 41
Appendix K: Head-to-Head Win Rate Matrix 42
Appendix L: Anomalies and Unexpected Patterns 43
L.1 Alpha-Beta(d=5) Second-Mover Wins 43
L.2 MCTS Scaling Non-Monotonicity 43
L.3 MCTS Self-Play FM Drop at 50,000 43
L.4 MCTS Performs Better Against Stronger Opponent 43
L.5 Alpha-Beta(d=8) Draw Discontinuity 43
L.6 Minimax(d=4) vs Greedy: 94% Gap 43
L.7 Machine-Dependent FM Rates 43
L.8 MCTS vs Greedy Sub-100% Ceiling 44
L.9 No Perfect 48-0 Games 44
Table L-1 44

1 INTRODUCTION

In 1726, Irish writer Jonathan Swift wrote a book titled *Guliver's Travels*, featuring an invention called The Machine, which Swift wrote with the purpose to act as an artificial intelligence (AI) and provide the scholars within the book with new ideas and research. However, not until almost two hundred years later, in 1914, did engineers apply AI to games. That year, Leonardo Torres y Quevedo, a Spanish engineer, demonstrated in Paris a fully automated machine capable of playing an endgame of chess between a rook and an opposing king. Torres Y Quevedo even set rules within the machine to only play legal moves and be able to detect illegal moves from its human opponent, signaling such action as an error. When started in a winning position, the machine reliably reached checkmate against human opponents without fault (Mucci, 2024).

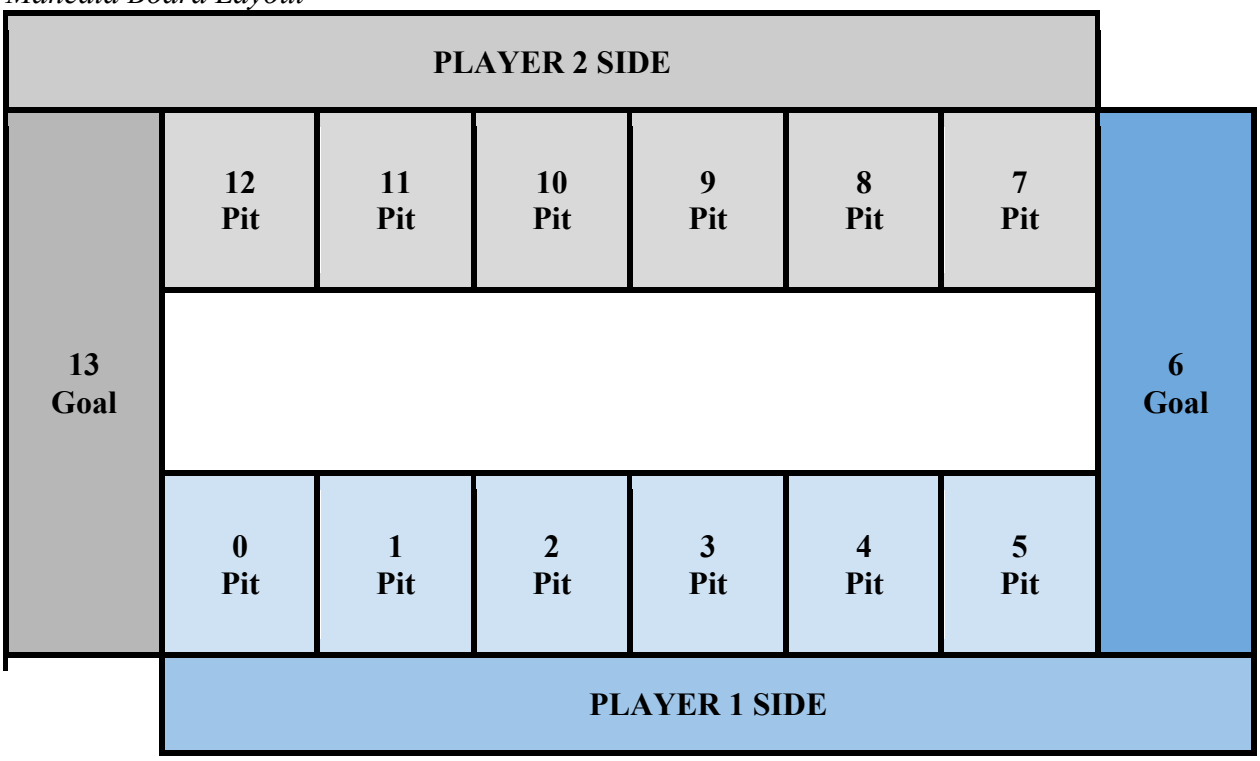
Later, in 1951, British computer scientist Christopher Strachey developed a checkers program to act as a domain for his machine intelligence experiments. Strachey initially programmed his checkers program on the Pilot Automatic Computing Engine and, due to memory constraints, later finished it on the Ferranti Mark 1 in the summer of 1952. Able to play a game of checkers from start to finish, the program operated at a "reasonable speed" and opened the door to implementing programs that complete complex, logical tasks through encoded rules and search. At the same time as Strachey in 1952, Arthur Samuel, an American computer scientist working at International Business Machines (IBM) also took on the challenge of programming an automatic checkers program. Samuel's program demonstrated core AI concepts of heuristics, evaluation functions, and search algorithms, leading to its consideration as one of the first successful self-learning programs in the world (Robinson, 2025).

In the context of AI, heuristics serve as guiding strategies and rules of thumb that assist in the decision-making process. Heuristics work hand-in-hand with search algorithms in order to provide solutions that are not guaranteed to be the most optimal but are nevertheless along a streamlined path. When implemented in games, AI uses heuristics to determine its next move, saving computational resources and reducing the need to search all nodes and their respective child nodes (Milvus, n.d.). Choosing which heuristic to use depends on the problem at hand.

Mancala is a family of ancient, two-player board games that dates back as far as 6110 BCE in East Africa, and its true origin remains unknown. This project uses a popular version of Mancala, called Kalah, for all rules, algorithms, and tests. Players set up the game by distributing an equal number of seeds, usually 48, amongst pits on a board such as Figure 1 below, leaving the goals, or Mancalas, empty. Following the setup, the two players alternate turns, selecting a set of stones on their respective sides and then traveling the stones in a counterclockwise direction around the board, dropping one stone until none remain in hand, leaving one in each pit, their own Mancala when encountered, and then in the pits of their opponent. Should the stone land in an empty pit on a given player's own side, the player then captures all of the stones adjacent to that pit on the opponent's side. If the last stone in a player's turn lands in their own Mancala, then the player is rewarded with another turn immediately. After either player clears all of the stones off their side of the board, the game ends, and the largest sum of stones in a player's own goal determines the winner (Savannah African Art Museum, n.d.; The Original Workshop, n.d.).

Figure 1

Mancala Board Layout



2 PROBLEM STATEMENT

The purpose of this project is to explore the implementation of AI in the game of Mancala which a multitude of AI algorithms and heuristics can accomplish. However, the standard twelve-pit board played in the Kalah version of the game presents a very large number of unique game states given the standard 48 stones to start and the complexity of the game itself. Previous research explores only a small portion of these game states, likely due to time and resource constraints. This replicates the models used in prior publications and builds upon the methods used in order to capture metrics covering a wider range of AI-driven games. As a result, the findings of this paper examine the existence of a threshold for diminishing marginal differences between AI agents as the number of games played vary and make assertions as to the limits of certain algorithms in the context of Mancala.

3 Challenges

Several technical and experimental challenges arise in the design, implementation, and evaluation of the AI agents for Mancala spanning the algorithm design, game-rule fidelity, computational cost management, parameter tuning, and experimental validity. The first challenge concerns heuristic evaluation function design. Depth-limited search agents cannot examine the full game tree and must instead estimate the quality of intermediate board states using a heuristic function. The quality of this heuristic directly determines the quality of the agent's play. Simple heuristics, such as the difference between the two players' store counts, are fast to compute but capture only one dimension of board quality. Gifford et al. (2008) tested seven heuristics for Mancala and found that performance varied widely, with multi-feature heuristics generally outperforming single-feature ones. However, richer heuristics introduce computational overhead

that may offset their informational advantage by reducing the depth the agent can search, represented by *d*, within a time budget. This creates a fundamental trade-off between evaluation sophistication and search depth, a tension that the literature identifies as a central design problem for game-playing agents (Ramanujan & Selman, 2011). An additional complication is weight calibration: when combining multiple features, the relative weights are set either manually or through an automated optimization procedure. Hunter (2021) used genetic algorithms for this purpose but found that feature selection mattered more than exact weight calibration, suggesting that hand-tuned weights are a practical alternative when automated methods exceed the project's scope.

Second, Mancala's extra-turn rule violates a core assumption of standard adversarial search algorithms. In most two-player games, players alternate moves, and algorithms like minimax exploit this by treating each tree level as belonging to the opposing player. In Mancala, when a player's last stone lands in their own store, that player moves again immediately. If the search algorithm fails to account for this, it will evaluate extra-turn branches as though the opponent is making the next move, producing inaccurate assessments of positions where extra turns are available. Rovaris (2017) identified this as an implementation concern specific to Mancala, and Irving et al. (2000) confirmed that turn continuation changes both game complexity and optimal strategies. The challenge extends to Monte Carlo Tree Search as well: researchers must adapt the standard backpropagation convention for games based on the AIMA reference implementation, assuming strict alternation when propagating simulation results up the tree where the same player may occupy consecutive tree levels. Section 5 describes the specific adaptations implemented for each agent type.

Third, the computational cost of adversarial search grows exponentially with depth. Each additional level of search multiplies the number of board states to evaluate by the branching factor. Gifford et al. (2008) reported that Mancala search depths above 10 required roughly 20 seconds per move, making deep search impractical for large-scale tournament evaluation. Because the experimental design requires thousands of games across dozens of agent pairings, even small increases in per-move computation compound into prohibitive total runtimes. Alpha-beta pruning addresses this by eliminating branches that provably cannot affect the outcome, reducing the effective branching factor without changing the result. The degree of improvement depends on move ordering: evaluating the strongest moves first increases the chance of early cutoffs. In the best case, move ordering reduces the effective branching factor to its square root (Gifford et al., 2008; Rovaris, 2017). This creates a practical asymmetry in the experimental design: the alpha-beta agent can be tested at greater depths than the minimax agent within the same time budget, enabling direct measurement of the efficiency gains from pruning.

Fourth, Monte Carlo Tree Search introduces its own tuning challenges, distinct from those of depth-limited search. MCTS does not rely on a heuristic evaluation function; instead, it estimates move quality through repeated random simulations (rollouts) to terminal states. The primary scaling dimension is the simulation budget: the number of rollouts performed before committing to a move. Performance generally improves with more simulations, but the relationship between budget and strength is nonlinear and opponent-dependent, making it difficult to predict in advance how many simulations an agent needs to compete with a given depth of adversarial search.

A second MCTS-specific challenge is the exploration constant C in the UCT selection formula, which governs the trade-off between exploitation (favoring well-tested high-reward moves) and exploration (sampling less-visited alternatives). The theoretical default of $C \approx \sqrt{2}$ is

derived from the upper confidence bound framework (Kocsis & Szepesvári, 2006), but Browne et al. (2012) note that the optimal value often requires domain-specific adjustment. Setting C too low causes premature convergence on suboptimal moves; setting it too high wastes simulations on unpromising branches. The experimental design addresses this through both budget sweeps at a fixed C and C sweeps at fixed budgets, separating the two sources of variation.

Fifth, Mancala exhibits a structural first-mover advantage that complicates experimental interpretation. Gifford et al. (2008) provided empirical evidence of this bias, and Irving et al. (2000) confirmed through solver analysis that the first player holds a winning advantage under standard Kalah starting configurations. Any tournament that does not control for seating position confounds agent strength with the arbitrary assignment of who moves first. The standard mitigation is a seat-swapping protocol in which each agent pair plays an equal number of games in each seating position (Lanctot et al., 2014).

A deeper challenge, less commonly addressed in the literature, is that the magnitude of the first-mover advantage is not constant across agent types. Stronger agents exploit the structural advantage more effectively than weaker ones, creating an interaction between agent intelligence and first-mover benefit. This means that overall win rates for strong-agent matchups can be misleading: they average together games where the first mover wins at very high rates and games where the second mover wins at very low rates, obscuring the degree to which outcomes are seat-determined rather than skill-determined. Accounting for this interaction requires reporting results split by seating position in addition to aggregate statistics, as discussed in Section 6.8.

Sixth, the agents in this study scale along fundamentally different computational dimensions, making direct comparison nontrivial. Greedy evaluates one move ahead with no search. Minimax and Alpha-Beta scale with search depth and heuristic quality. MCTS scales with simulation budget and is further influenced by the exploration constant and rollout policy. No single metric captures all relevant dimensions of agent performance. Win rate alone is confounded by opponent strength distribution and first-mover effects. Computation time alone ignores quality. A meaningful comparison requires multiple complementary metrics, including win rate with confidence intervals, score differentials, normalized computation time, and a ranking system such as Elo that adjusts for opponent quality (Elo, 1978; Coulom, 2008).

Finally, the Greedy agent introduces a minor challenge related to nondeterminism. The agent selects the move that maximizes immediate store gain, but when multiple moves produce the same gain, ties are broken randomly. This means that the Greedy agent's results exhibit slight variance across runs even with identical configurations, requiring sufficient sample sizes to produce stable estimates. While this variance is small relative to differences in agent strength, it is important to account for when interpreting results involving the Greedy baseline, particularly in small-sample matchups.

3.1 Related Works

One important direction in Mancala research is classical adversarial search, where the AI looks ahead at possible future moves while assuming that the opponent also plays well. A standard method in this area is Minimax, which expands a search tree and chooses moves by alternating between maximizing and minimizing outcomes. In Mancala, this allows the agent to evaluate future board states and select the move that leads to the best result under rational play. Gifford et al. (2008), Hunter (2021) and Rovaris (2017) describe this general search-based approach.

A common improvement to Minimax is implementing Alpha-Beta pruning, which does not change the final decision, but instead makes the search more efficient by skipping branches that

cannot affect the end result. As Gifford et al. (2008) and Rovaris (2017) described, this becomes very important when search depth increases, because even a relatively simple game like Mancala can become computationally expensive. Research on Mancala (Irving et al., 2000) shows that search can become much stronger when combined with techniques such as move ordering, transposition tables, futility pruning, and endgame databases.

Another key issue in Minimax-style methods is the evaluation function because it is usually not possible to search all the way to the end of the game, the agent must estimate the quality of a board state at the cutoff depth. Gifford et al. (2008), Hunter (2021) and Rovaris (2017) show that heuristic design is central to strong Mancala play. The literature includes both simple heuristics, such as the difference between the players' stores (Hunter, 2021; Rovaris, 2017), and richer ones based on factors like score advantage, proximity to winning, stone distribution, available moves, limiting the opponent's gain, and the chance of earning extra turns (Gifford et al., 2008; Hunter, 2021; Rovaris, 2017). This is directly relevant to the present project, since the Minimax and Alpha-Beta agents in this project also rely on an evaluation function. The papers also suggest that stronger heuristics can improve performance, although they often increase complexity and computational time.

A diverging direction in the literature is MCTS because unlike Minimax, MCTS relies less on a handcrafted evaluation function and instead uses repeated simulations to estimate which moves are viable and rewarding. Lanctot et al. (2014), Ramanujan & Selman (2011), and Rovaris (2017) discuss the MCTS approach as an alternative to traditional tree search with UCT as a common MCTS variant that balances exploring moves that have performed well so far with trying moves that have been sampled less often in order to minimize the large search space of Mancala while. The authors also draw comparisons between the Minimax-style search and UCT in Mancala to show that both algorithms can perform well in different ways. Their results suggest that Minimax depends more on search depth and evaluation quality whereas MCTS performance is more dependent on simulation budget and rollout policy. In this context, simulation budget refers to the amount of computation allocated to MCTS before it selects a move, which may be expressed in terms of the number of simulations, playouts, iterations, or nodes that the algorithm expands. Ramanujan and Selman (2011) show that, in Mancala, performance under UCT is dependent on the total search effort available and on how that effort is allocated. They find that increasing the number of UCT iterations can be more effective than allocating the same computational budget to additional playouts per leaf. Their findings also suggest that pure UCT can be quite strong under a fixed node budget. However, at the same time, they show that heuristic Minimax remains highly competitive, and in some cases may outperform UCT approaches that rely more heavily on playout-based estimation.

The literature also includes hybrid methods that combine ideas from both approaches. Lanctot et al. (2014) and Ramanujan & Selman (2011) discuss systems that add heuristic guidance to simulation-based search. For example, Lanctot et al. (2014) proposes storing Minimax-style heuristic values alongside simulation statistics, and Ramanujan & Selman (2011) shows that adding a board evaluation function to UCT and updating the search tree in a Minimax-style way can outperform both standard Alpha-Beta and plain UCT. This approach uses max and min operations to pass heuristic information upward through the tree according to which player is making the decision. These results are important because they show that stronger Mancala agents often combine simulation-based search and heuristic guidance. However, such methods may be too complex for a basic course project unless there is extra time.

These papers inform the present work in several ways. First, they confirm that Mancala is a good domain for studying AI search methods and show that both Minimax-based search and MCTS-based search can perform well. This also makes Mancala a useful setting for comparing different search strategies. At the same time, the papers help set realistic expectations. Some of the strongest Mancala systems use techniques that go beyond a basic class project, such as transposition tables, futility pruning, endgame databases, improved simulation policies, or hybrid methods that combine MCTS with Minimax-style evaluation (Irving et al., 2000; Lanctot et al., 2014; Ramanujan & Selman, 2011). These methods show what stronger systems can look like, but they also suggest that reproducing state-of-the-art performance would require much more engineering effort than implementing the core algorithms alone.

3.2 Importance and Impacts

Mancala is an ideal setting for studying several core principles in AI including, but not limited to, adversarial search, heuristic evaluation, and simulation-based decision-making. The project examines practical issues such as extra turns, evaluation function design, computational cost, and first-player advantage in a controlled environment. There is also valuable from an educational and methodological perspective because board games have long been used in AI research due to their rule sets, measurable outcomes, and repeatable testing under the same conditions. Likewise, Mancala is especially suitable for such research since its rules are simple enough to fully implement while still creating meaningful strategic challenges such as captures, extra turns, and a growing search space. This project also shows the importance of comparing AI methods fairly rather than relying on only one performance measure through the method of seat-swapping in tournaments, move timing, win-rate summaries, score differentials, and visual analysis.

Beyond games, this project uses methods that are relevant to other areas of computer science and decision-making. Minimax and alpha-beta pruning algorithms such as the ones in this work represent planning through explicit lookahead strategies, while Monte Carlo Tree Search (MCTS) represents a simulation-based approach that can be useful when exhaustive search is too expensive. Similar trade-offs appear in domains such as robotics path planning, where agents must choose between exhaustive search and real-time approximation, medical treatment sequencing, where sequential decisions must weigh computational cost against outcome quality, and supply chain optimization, where simulation-based methods complement analytical planning. Because of this, the project helps illustrate when a simple method may be enough, when deeper search is worth the extra time, and when simulation-based methods may be more practical.

In terms of social or business aspects, the project does not directly address a commercial problem, so its immediate business impact is limited. However, the research still has value in showing that AI systems can be designed and evaluated carefully. Skills such as modeling rules correctly, choosing appropriate evaluation metrics, and testing algorithms under controlled conditions are important far beyond games.

4 DATA COLLECTION AND PREPROCESSING

4.1 Data Collection

The project data comes from simulated games played in a Python implementation of Mancala which represents each board state as a 14-element tuple composed of six pits and one store for each player which also keeps track of the current player, available legal moves, and whether the game has reached a terminal state. This type of data is appropriate for the project

because the goal is to compare how different search agents perform under the same game rules and conditions. Data collection occurs by running tournaments between the different agent configurations of random, greedy, Minimax agents at depths 4 and 6, Alpha-Beta agents at depths 3 through 8, and MCTS agents with simulation budgets of 1,000 through 100,000 iterations per move. For each matchup, the engine plays games in both seating positions so that each agent has a chance to play first and second in order to produce a dataset of game outcomes and performance measures that explores first-move advantage. For each game, each game record includes final scores, score differential, total moves, moves per player, average time per move, captures, extra turns, and whether the first player won. These measures are directly relevant because they show both how strong an agent is and how much computation the simulation requires. In addition to per-game data, the project also produces summary measures at the matchup and agent level, such as win rate, average score differential, and average computation time per move.

For exploratory analysis, the most important step is to summarize and compare the outcomes across agents. Descriptive statistics such as the mean, standard deviation, minimum, maximum, and range can be reported for score differential, move count, captures, extra turns, and computation time whereas Wilson score confidence intervals accompany all reported win rates. Visualizations such as bar charts, histograms, boxplots, and heatmaps show the differences between agents, and a scatterplot compares the extra-turn rate and win rate.

4.2 Preprocessing

Preprocessing involves checking consistency and comparing performance fairly because the data is produced by a rule-based game engine, so missing values should not normally occur. Instead, preprocessing focuses on checking for invalid game states, such as negative pit values, invalid moves, incorrect total stone counts, or mismatches between the winner and the final scores. It is also important to ensure that seat-swapped games are reported consistently, following the methodology described by Lanctot et al. (2014),

Some variables need careful interpretation such as computation time per move since it is generally right-skewed, especially for deeper Minimax searches or larger MCTS budgets. Therefore, a log-scaled plot interprets the results in order to mitigate such imbalances. Standard scaling or normalization is not required for the tournament analysis itself since the project is not training a predictive model. However, normalization is still useful inside the evaluation function so that heuristic values stay on a comparable scale. In the context of a Mancala game engine, unusually long games, very large score differences, or large time spikes may reflect real behavior of the algorithms rather than errors in the data, and so as a result, outliers remain in the datasets unless they clearly result from implementation problems. In the same way, imbalance in outcomes, such as one agent winning much more often than others, is treated as part of the result rather than corrected artificially. There is no need for data augmentation in the usual machine learning sense since the project does not depend on a limited training sample. Instead, the equivalent is to run more games and include both seating positions so the results are more stable and representative.

5 METHODOLOGY

5.1 Heuristic Evaluation Function Design

Due to the Minimax and Alpha-Beta pruning agents evaluating board states at a depth cutoff rather than searching to the varying depths of the terminal states, the quality of the evaluation function directly determines the quality of the agents playing the game. In a terminal state, the game engine returns the true utility, computed as +1, −1, or 0 based on the store differential, but at non-terminal cutoff nodes, the agent must estimate the respective states' values using a heuristic. The evaluation function in this project assigns a numeric score to each non-terminal board state by combining several features identified in the literature as predictive of Mancala outcomes.

The current codebase uses a simple evaluation function that calculates a single normalized store difference.

$$\frac{P1_{store} - P2_{store}}{Total\ Stones} \quad (1)$$

Equation (1) gives a fast, easy-to-read estimate that always falls between −1 and +1 regardless of how many stones remain. While this works as a starting point, the evaluation function only measures one part of board quality, and studies show that using more features is more effective (Gifford et al., 2008; Hunter, 2021).

Rather than computing static proxies for capture potential or extra-turn opportunities, which require assumptions about which moves a player might choose, a richer evaluation function performs a single-move lookahead: for each legal move available to each player, it executes the move on a hypothetical copy of the board state and records the actual store gain and whether an extra turn results. With at most six legal moves per side, this simulation cost is minimal.

The rich evaluation function combines four features.

$$h(s) = 0.5x_1 + 0.1x_2 + 0.2x_3 + 0.2x_4 \quad (2)$$

In Equation (2), x_1 is the normalized store differential identical to Equation 1, x_2 is the difference in stones between the current player and their opponent, x_3 is the average number of stones added to the store per available move, and x_4 is the difference between the number of possible moves yielding an extra turn for the current player and their opponent. The store differential receives the largest weight of 0.5 because it directly measures progress toward the win condition, whereas the average store gain and extra-turn count each receive a weight of 0.2 due to their roles as tactical indicators of position quality, and the player-side-stone differential receives the smallest weight of 0.1 to serve as a positional tiebreaker. The authors hand-tuned the weights based on the relative importance suggested by prior empirical results (Gifford et al., 2008; Hunter, 2021). Hunter (2021) uses genetically optimized weights for a similar heuristic but notes that feature choice matters more than exact weight calibration, supporting hand-tuned weights as a practical alternative. Both evaluation functions test at matched search depths to measure the marginal benefit of richer position assessment.

For the MCTS agent, no heuristic evaluation function is used because MCTS evaluates positions through random rollouts, meaning that from a given leaf node, the algorithm selects moves uniformly at random until the game terminates and returns the terminal utility. While heuristic rollout policies can provide improvement to MCTS performance in some domains (Rovaris, 2017), this project retains the standard random policy to isolate the effect of simulation budget on play quality without confounding it with rollout policy design.

5.2 Game Environment

The Mancala game engine implements the Kalah configuration of six pits per player, four stones per pit, and one store per player, for a total of 48 stones, and so the board state is represented as a tuple of 14 integers. Indices 0 through 5 correspond to Player 1's pits from left to right, index 6 is Player 1's store, and indices 7 through 12 are Player 2's pits sown left to right from Player 2's perspective with index 13 being Player 2's store. State representation in the game engine also includes the current player, the terminal utility, and the set of legal moves, packaging the components into immutable state objects that ensure the search algorithms can freely copy and modify states without side effects.

During a player's turn, the engine distributes stones counterclockwise and skips the opponent's store. The extra-turn rule triggers when the last stone lands in the current player's store. In this case, the current player remains unchanged in the resulting state. Additionally, the capture rule triggers if the last stone lands in an empty pit on the current player's side and the opposite pit contains stones. Both the landing stone and the opposite pit's stones move to the player's store. When either player clears their own side, the game ends, and the engine moves all remaining pit stones to each respective store, clearing the pits, and returning to a terminal state. Utility is +1 for a Player 1 win, −1 for a Player 2 win, or 0 for a stalemate depending on the store difference.

5.3 Agent Descriptions

5.3.1 Random Agent

Random agents select uniformly at random from the set of legal moves at each turn and do not use an evaluation function or search function. This agent serves as a lower-bound baseline, representing the performance of a player with no strategic knowledge.

5.3.2 Greedy Agent

Greedy agents only look at a single move ahead of their current state, selecting the legal move that maximizes immediate store gain and does not anticipate opponent responses or future consequences, serving as a baseline for evaluating the benefits of deeper search. The algorithm randomly breaks ties between equally good moves with the equivalence of tossing a coin.

5.3.3 Minimax Agent

Minimax agents perform adversarial searches limited to depths of four or six depending on the match setup, choosing the move that maximizes its own gain while assuming that the opponent is working to minimize that same gain. At terminal nodes, the minimax agent uses the true game utility by applying the heuristic evaluation function at the depth cutoff. A critical adaptation for Mancala is the depth extension for extra turns because when a move results in the same player moving again, extra-turn sequences are within the node search depth limit, preventing the minimax agent's search from treating an extra turn as taking another turn, which could otherwise result in the agent undervaluing extra-turn moves. Similar to greedy agents, minimax ties between moves with equal evaluation scores are broken randomly to avoid systematic low-index pit bias.

5.3.4 Alpha-Beta Agent

Alpha-Beta agents implement the minimax search algorithm with pruning logic and move ordering. The pruning logic maintains the best score that the maximizer can guarantee, called alpha, and the best score that the minimizer can guarantee, called beta, cutting off subtrees and child nodes that cannot influence the final decision due to not exceeding either of these values. Implementing this agent reduces the number of nodes evaluated, subsequently reducing the time

spent searching and reducing the overall computation budget without changing the selected move at any given depth. To maximize pruning efficiency, the alpha-beta agent applies move ordering before evaluating the children at each node. Two criteria sort the moves: extra-turn moves and immediate store gain, with a random component to break ties. Ordering ensures that the strongest candidate moves are evaluated first, increasing the likelihood of early cutoffs. As with the Minimax agent, the alpha-beta agent applies depth extensions for extra turns. The tournament setup tests the alpha-beta agents tested at depths of four, six, and eight in order to enable direct comparison of computation time savings from pruning while simultaneously measuring whether move ordering affects decision quality, defined as an agent's consistency in selecting optimal moves in a fast manner, relative to standard Minimax.

5.3.5 Monte Carlo Tree Search Agent

MCTS agents use UCT to guide search based on the AIMA reference implementation with modifications for the non-alternating turns in Mancala. The first phase in each iteration, selection, traverses the tree from the root, choosing children according to the Upper Confidence Bound (UCB)1 formula.

$$UCB(n) = \frac{U}{N} + C\,\frac{\sqrt{ln(N_{parent})}}{N} \tag{3}$$

In Equation (3), U is the cumulative reward, N is the visit count, C is the exploration constant which by default is $1.4 \approx \sqrt{2}$, N_{parent} is the visit count of the parent node, and C is the exploration constant (default $1.4 \approx \sqrt{2}$). The first term (U/N) favors nodes with high average reward; the second term grows when a node has been visited infrequently relative to its parent, encouraging the algorithm to try under-explored branches. The exploration constant C controls the balance between these two pressures. Kocsis and Szepesvári (2006) derived the theoretical default from the upper confidence bound framework and demonstrated its robustness across simulation budgets. The MCTS agent tests at simulation budgets of 1,000, 2,000, 5,000, 10,000, 20,000, 50,000, and 100,000 iterations per move and selects the root child with the highest visit count. Nodes with zero visits receive infinite UCB scores to ensure that the agents sample all children at least once. Next, the second phase, expansion, adds all legal children of a reached leaf node, shuffling the children randomly before adding in order to avoid systematic pit-index bias in initial UCB tie-breaking. Third, simulation executes a random rollout from the expanded node to a terminal state, selecting legal moves uniformly at random and returning the game utility from the root player's perspective. Finally, backpropagation, the fourth phase, updates the visit counts and cumulative rewards along the path from the expanded node to the root. Utility mapping converts the raw result so that draws are valued between losses and wins rather than only crediting wins and combining losses.

5.4 Experimental Protocol

The evaluation protocol uses a structured pairwise tournament that tests 30 agent configurations through a combination of the random and Greedy agents, Minimax agents at depth limits of four and six nodes, alpha-beta agents at depth limits of three to eight nodes,, and MCTS agents at simulation budgets of 1,000 through 100,000 iterations with alternative C parameter values tested at selected budgets. Together, these configurations produce 117 unique matchups including self-play matchups for first-move advantage analysis.

For each matchup, the engine runs a set number of games with one agent A moving first and then the same number of games with the other agent in the pair moving first, ensuring balanced exposure to the first-mover advantage. A 40-column CSV format captures the statistics of each game, including but not limited to agent identities, parameters, scores, move counts, computation times , capture counts, extra-turn counts, first-mover status, and a timestamp. The default number of games is 25 games per seating resulting in 50 total games per matchup, with key matchups running at higher counts such as 100 to 200 games in order to provide tighter confidence intervals.

To enable parallel data collection across multiple machines with different hardware, the game engine computes a deterministic benchmark at the start of each simulation batch equivalent to the time for an alpha-beta with a depth limit of six to search the initial board state, averaged over 10 repetitions. Recording this benchmark alongside each game's data allows post-hoc normalization of computation times to a common scale. Additionally, the CSV records machine identifiers to distinguish data sources.

The final dataset comprises 9,408 games after deduplication of 100 duplicate records, collected across multiple simulation batches. All core pairwise matchups include at least 50 games, with key matchups at 150 to 200 games. Three primary metrics are reported: win rate with Wilson score 95% confidence intervals, average score differential, and normalized computation time per move.

5.5 Software Architecture

Organizing this project as a single-module Python file allows for seamless interactive exploration with a corresponding Jupyter notebook. Seven distinct sections comprise the module: the AIMA framework classes, board constants, the Mancala game class, evaluation functions, player implementations, the game runner and CSV infrastructure, and the tournament interface.Providing three primary nodes, the command-line interface supports an interactive menu with options to play, watch, or run matchups, a tournament flag for automated tournament execution, and a CSV flag to enable game-level data collection. Defining the agent catalog as a dictionary mapping labels to functions simplifies the addition of new agent configurations without requiring modifications to the tournament logic.

6 Results and Interpretation

The results consist of 9,408 games played across 117 matchup configurations involving different 30 agent configurations. Furthermore, the timing data is normalized across two machines using a rigorous benchmark calibration procedure to ensure hardware consistency. We report the win rates of the agents with 95% Wilson confidence intervals, while binomial tests assess the statistical significance of the win rate against a 50% null hypothesis which determines if the observed binary outcome differs significantly from a 50/50 distribution. Cohen's h quantifies the practical magnitude of observed differences in proportions, where $|h| < 0.2$ indicates a small effect, 0.2 to 0.8 a medium effect, and values above 0.8 a large effect. Complete data, formulas, and charts

are provided in the companion Excel workbook. Statistical tests, Elo ratings, regression models, and predictive analyses are documented in the companion Jupyter notebook.

Table 1

Game Analysis Data

Item	Value
Total games	9,408
Matchup configurations	117
Agent configurations	30
Seat protocol	Equal games as Player 1 and Player 2
Timing normalization	Benchmark-calibrated across 2 machines
Win-rate uncertainty	95% Wilson confidence intervals
Significance test	Binomial test vs. 50% null
Effect size	Cohen's h

6.1 Agent Hierarchy

To produce a unified ranking of relative skill across all 30 agent configurations, we use an Elo rating system. Elo rating is a pairwise comparison method originally for chess (Elo, 1978) that is now appearing more frequently in the research of AI zero-sum games (Coulom, 2008). Each agent begins at a score of 1500 by default, and after each game, the winning agent gains a number of rating points. Conversely, the losing agent loses the same number of rating points, with the magnitude of the change in points matching how surprising the outcome was given the current rating gap. Because Elo ratings depend on the order in which games , the analysis shuffles and processes the full set of 9,408 games 20 times. The final ratings average all 20 runs, computing bootstrap 95% confidence intervals from the distribution of ratings each run. An Elo rating difference of 400 points corresponds to about a 10:1 expected win ratio for the player with the higher rating, so the gaps between top agents are relatively small in practical terms. See Appendix A for the complete table and a discussion of Elo methodology.

Alpha-Beta agents with a search depth of eight and MCTS agents playing 100,000 simulations have the top two Elo ratings in this project of 1716 and 1724 respectively. Alpha-Beta agents with a search depth of six hold the third highest score of 1694. The gap between the two highest alpha-beta agents is only 22 Elo points despite a ninefold difference in computations per move, suggesting that a search depth of six is the practical strength ceiling for this game variant. Below the top three agents are MCTS agents with relatively moderate budgets of 20,000 to 50,000 simulations and occupy the 1570 to 1670 Elo rating range, while alpha-beta agents with a four-node depth search and minimax agents have ratings of 1530 to 1580. Coming in last are the greedy and random agents with ratings of 928 and 595 respectively.

Figure 2

Top Five AI Agents by Elo Rating with 95% Confidence Interval

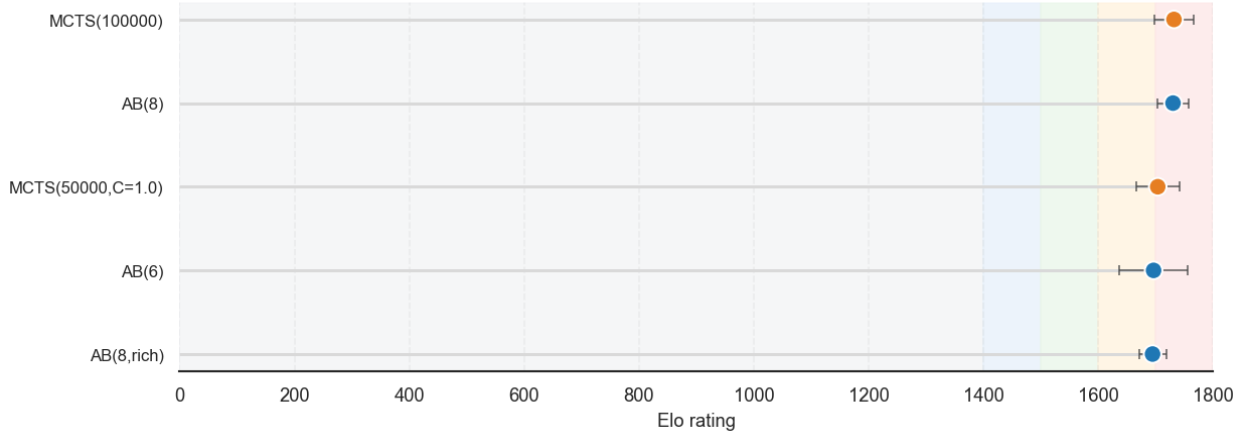
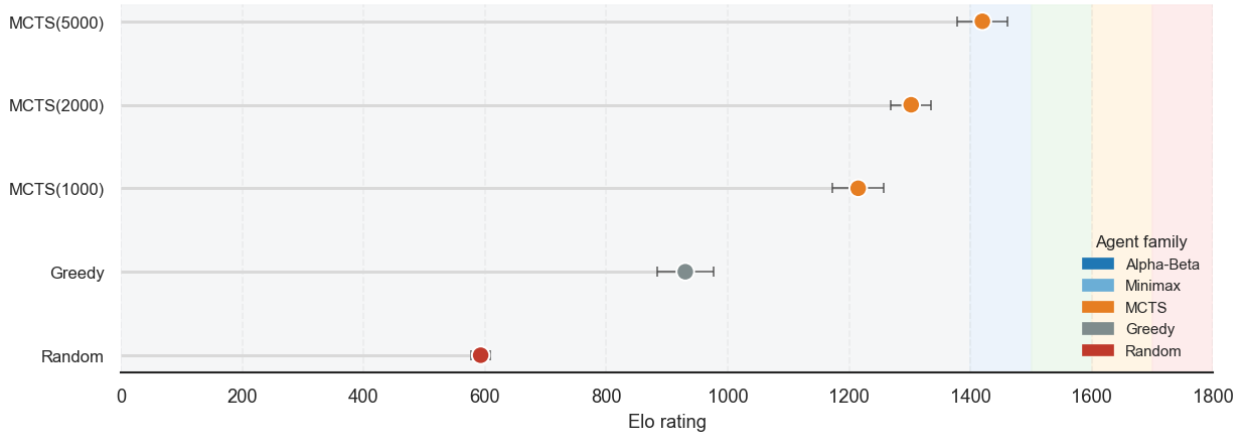


Figure 3

Bottom Five AI Agents by Elo Rating with 95% Confidence Interval



6.2 Alpha-Beta Pruning vs. Minimax

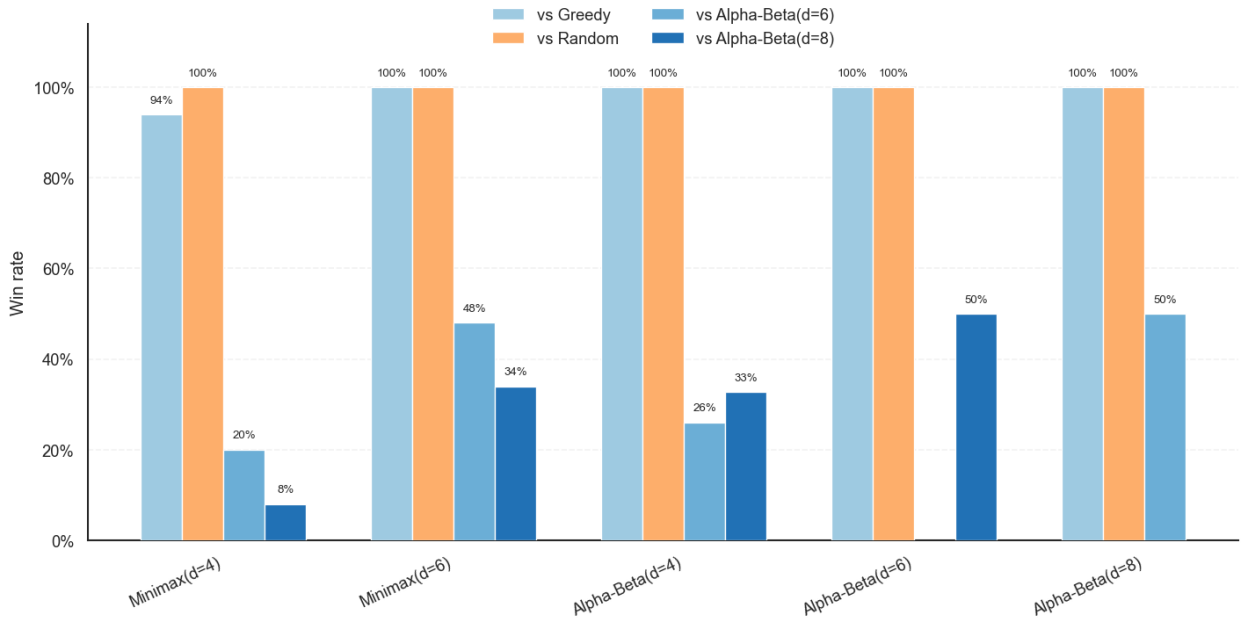
Agents executing alpha-beta pruning produce substantial speedups over standard minimax with no meaningful change in competitive strength. At a node depth of four, alpha-beta averages 0.006 ms per move compared to the 0.018 ms that Minimax agents take on an average turn, a speedup of three-fold. The head-to-head record was 27 wins for Minimax, 22 wins for Alpha-Beta, and one draw in 50 games with a binomial test p equal to 0.67 and a Cohen's h equal to 0.08, . When increasing the depth-search limit to six nodes, the difference between search times become 0.68ms per move for alpha-beta agents and 1.071ms per move for minimax agents, a much larger speedup of 15.7-fold, with the head-to-head resulting in 24 wins for alpha-beta agents, 25 wins for minimax agents, and one draw($p = 0.89$, $h = -0.04$). Both effect sizes fall well below the $|h| < 0.2$ small-effect threshold defined in Section 6's opening paragraph, confirming the theoretical expectation. In this implementation, the two algorithms are not strictly decision-identical because alpha-beta agents incorporate move ordering that prioritizes extra-turn moves and sorts remaining moves by store gain. Despite this ordering advantage, competitive outcomes remain statistically equivalent, suggesting that move ordering primarily improves efficiency rather than decision quality at these depths.

6.3 Effect of Search Depthd

Deeper search consistently improved agent strength against weak opponents. Alpha-Beta(d=4), Alpha-Beta(d=6), and Alpha-Beta(d=8) each achieved a 100% win rate against both Greedy and Random across 50 games per matchup. Minimax(d=4) won 94% against Greedy, while Minimax(d=6) won 100%. However, increasing depth beyond 6 yielded diminishing returns in head-to-head competition between strong agents. Alpha-Beta(d=6) and Alpha-Beta(d=8) split 100 games exactly 50 to 50 with zero draws. This apparent parity is entirely seat-determined: every one of the 100 games was won by whichever agent moved first, and the seat-swapping protocol assigned each agent exactly 50 first-mover games. The two agents are competitively identical once seating is controlled. Alpha-Beta(d=8) required roughly nine times more computation per move (0.591 ms versus 0.068 ms) for zero additional competitive benefit, confirming that a search depth of six is the practical ceiling for this game variant. The depth extension, which does not consume search depth on extra-turn continuations, effectively gives alpha-beta agents access to deeper tactical sequences than the nominal depth parameter suggests. This may explain why the practical ceiling occurs earlier than theoretical branching-factor analysis would predict. Detailed results appear in Appendix H.

Figure 4

Depth Scaling Against Weak and Strong Opponents



6.4 MCTS Simulation Budget and Scaling

MCTS performance scales with simulation count, but the rate and ceiling of improvement depends on the strength of the opponent. Against Greedy agents, MCTS agents achieve high win rates even at modest simulation budgets: 88% at 1,000 simulations and 98% at 10,000 simulations. Against Alpha-Beta opponents, MCTS agents require substantially more simulations to compete. Table 1 presents the MCTS scaling results against Alpha-Beta.

Table 2

MCTS win Rate Against Alpha-Beta by Simulation Budget

Simulations	vs AB(d=4)	Games	vs AB(d=6)	Games	vs AB(d=8)	Games
1,000	26%	50	6%	50	0%	50
2,000	18%	50	12%	50	10%	50
5,000	49%	100	10%	100	2%	50
10,000	49%	150	22%	200	14%	200
20,000	46%	200	28%	150	31%	150
50,000	58%	200	62%	150	37%	150
100,000	58%	200	50%	200	47%	150

The crossover point where MCTS exceeds 50% against Alpha-Beta(d=6) occurs between 20,000 and 50,000 simulations. Against an Alpha-Beta with a search depth of eight, MCTS agents at 100,000 simulations reach 47%, approaching but not exceeding parity. Logistic regression models that fit this data predict that MCTS agents need a simulation budget of approximately 500,000 in order to achieve 80% against Alpha-Beta agents with a search depth of six and approximately a larger simulation budget of 1,000,000 to approach only 82% against Alpha-Beta agents with a depth-search limit of eight. Appendix B provides the complete MCTS scaling matrix with confidence intervals and corresponding charts.

6.6 Evaluation Function

At depth 4, the standard evaluation function won 28 of 50 games against the rich variant, with 6 draws ($p = 0.48$), suggesting that the simpler heuristic retains a slight edge at shallow search. At depths 6 and 8, both matchups were tied at 25 to 25 ($p = 1.00$), indicating convergence at deeper search. A knowledge-versus-search comparison pitting Alpha-Beta(d=4,rich) against Alpha-Beta(d=6,standard) yielded 11 to 28 with 11 draws ($p = 0.009$, $h = -0.61$), confirming that additional search depth generally outweighs evaluation sophistication in this domain (Ramanujan & Selman, 2011). Complete results appear in Appendix D.

6.7 Extra Turns as a Tactical Mechanism

Extra turns emerged as the dominant tactical mechanism during the Mancala gameplay. Across all 8,901 decided games, winners averaged 9.03 extra turns compared to 3.64 for losers , a ratio of 2.5 to 1. A multiple regression predicting score for an opponent based on extra turns, captures, and total moves yielded a coefficient of determination of 0.476, with extra turns contributing the most to that coefficient. Notably, when winning, Alpha-Beta agents averaged 12.74 extra turns per game whereas MCTS agents averaged 5.56 extra points per game, reflecting the Alpha-Beta agent's ability to plan through multi-step extra-turn sequences using the depth extension. See Appendix E for the full analysis, including a one-way ANOVA across algorithm families.

6.8 First-Move Advantage

The present data confirms that the first-move advantage in Mancala reveals that its exploitation scales with agent strength, creating what this paper terms an intelligence gradient.Across all 8,901 decided games, the first mover won 74.3%,, dispelling chance as a factor.In self-play experiments when an agent of one type plays against another of the same type, the first-mover win rate among decided games ranged from 54% for MCTS at 1,000 simulations to 100% for all Alpha-Beta agents at search depths of three to eight. Table 2 presents the self-play first-mover results.

Table 3

Self-Play First-Mover Results

Agent	Elo	Decided	FM Wins	SM Wins	Draws	FM Rate	95% CI
Greedy	928	92	62	30	8	67%	[57%, 76%]
MCTS(1k sims)	1220	90	49	41	10	54%	[44%, 65%]
MCTS(5k sims)	1410	92	63	29	8	68%	[58%, 77%]
MCTS(10k sims)	1510	142	114	28	8	80%	[73%, 86%]
MCTS(20k sims)	1569	96	80	16	4	83%	[75%, 90%]
AB(d=4)	1574	100	100	0	0	100%	[96%, 100%]
AB(d=6)	1694	100	100	0	0	100%	[96%, 100%]
AB(d=8)	1716	56	56	0	44	100%	[94%, 100%]

The pattern suggests that Mancala's first-mover advantage is structural but conditional on agent skill, meaning that weak agents cannot reliably detect and exploit the tactical opportunities that moving first affords, so their first-mover rate ranges from 54% to 67%. Stronger agents, particularly Alpha-Beta agents, consistently convert the first-mover position into a win. At a search depth of eight the draw rate jumped to 44% from 0% with Alpha-Beta agents that had a search depth less than eight, with all draws resulting in 24 points for each player. Among decided games at depth eight, the player with the first move won all 56. As far as moves for how each game ended, drawn games averaged 27.9 moves compared to 42.8 for decided games, suggesting that both agents detect the forced-draw path early. This threshold effect likely represents partial convergence toward the theoretical equilibrium identified by Irving et al. (2000). See Appendix F for the full first-mover analysis.

7 Discussion of Results

7.1 Summary of Findings

The experimental results address the project's core research questions, beginning with an analysis of how much Alpha-Beta pruning improves efficiency over standard Minimax agents and

its impact on decision quality. Alpha-beta pruning achieved speedups of 3-fold to 16-fold over minimax with no competitive penalty, confirming the theoretical guarantee that pruning preserves decision quality, aligning with Gifford et al. (2008) who reported similar results in their Mancala experiments. In investigating how search depth affects competitive strength and the location of a practical ceiling, the results show that deeper search improved performance against Random and Greedy agents up to a practical ceiling at depth six, beyond which Alpha-Beta agents offered no additional competitive advantage despite ninefold greater computation. The study also determined how many MCTS simulations are needed to compete with depth-limited adversarial search, finding that MCTS agents competed with Alpha-Beta agents only at high simulation budgets, requiring approximately 50,000 simulations for parity with Alpha-Beta agents with a search depth of six. Throughout these tests, the default exploration constant C = 1.4 proved robust across budgets, consistent with the theoretical derivation by Kocsis and Szepesvári (2006). When evaluating whether a richer evaluation function improves play or if deeper search matters more, the data indicated that while a rich evaluation function achieved parity with the standard heuristic at depths six and above, deeper search consistently outperformed richer evaluation, supporting the general principle that search depth dominates evaluation sophistication in deterministic, fully observable games (Ramanujan & Selman, 2011). Finally, the results identified the tactical mechanisms driving victory and the interaction between structural first-mover advantages and agent intelligence. Extra turns emerged as the dominant tactical mechanism, with winners earning 2.5 times more extra turns than losers. First-mover advantage was universal at 74.3% and its effectiveness scaled sharply with agent intelligence from 54% to a perfect 100% win rate.

7.2 The Intelligence and First-Mover Relationship

Perhaps the most noteworthy finding is the relationship between that first-mover exploitation scales with agent strength across all tested algorithm families. Mancala's first-mover advantage is well documented (Irving et al., 2000; Gifford et al., 2008), but prior work has not systematically examined how exploitation magnitude varies with agent strength across multiple algorithm families. The present results show a clear gradient depicting that the first-mover win rate in self-play increases from 54% for MCTS agents checking 1,000 simulations, barely above chance, to 100% for Alpha-Beta agents at all depths, among decided games. Theoretically, this gradient illustrates how a structural feature of the game such as the first-mover advantage becomes increasingly exploitable as agents improve their ability to plan multi-step tactical sequences. Practically, the gradient means that stochastic agents such as the MCTS and Greedy agents, which rely on random rollouts or single-move evaluation rather than deep search, fail to convert a structural advantage that the minimax and alpha-beta depth-limited search agents exploit consistently. Examining the gap between the MCTS agents' exploitation of first mover advantage 67% to 83% of the time compared to the Alpha-Beta agent's 100% use of the exploitation suggests that the MCTS agent's random rollouts fail to consistently identify the multi-step extra-turn sequences that the first mover can use to their advantage. Therefore, a heuristic-guided rollout policy, as suggested by Rovaris (2017), might close this gap.

The extra-turn data support the claim that MCTS agents fail to exploit extra-turn sequences as effectively as alpha-beta agents.. Alpha-Beta agent winners averaged 12.74 extra turns per game, nearly 2.3 times the MCTS average of 5.56, indicating that the first-mover advantage is mediated by deliberate extra-turn planning rather than incidental turn accumulation. Extension of the node search depth limit amplifies this effect because the extra turns do not consume search

depth, allowing Alpha-Beta agents to evaluate chains of consecutive moves that would otherwise be invisible to shallower searches.

7.3 Relationship to Prior Work

Consistent with Gifford et al. (2008), Alpha-Beta agents dominate at moderate depths when using store-focused heuristics. Moreover, the MCTS agent scaling pattern aligns with Rovaris (2017), who found that MCTS agents require substantial simulation budgets in order to match the heuristic-based searches of other agents playing Mancala. Regarding first-mover outcomes, Irving et. al. (2000) previously demonstrated through solver analysis that the first player holds a winning advantage under standard starting configurations. By quantifying the extra-turn mechanism as the primary driver of first-mover advantage, the present work adds a level of statistical detail that is missing from prior literature. This knowledge-versus-search result, where deeper search beats richer evaluation, mirrors broader findings in game-based AI research (Ramanujan & Selman, 2011), where search depth tends to dominate heuristic quality in deterministic, fully observable games with moderate branching factors.

7.4 Practical Implications

These results highlight several practical implications for AI system design that may help researchers to select build optimal game engines and implement more accurate benchmark standards. First, Alpha-Beta agents with a search depth of six reach the Pareto-optimal balance between strength and computation for Mancala at 0.068 ms per move. Developers of real-time Mancala applications should adopt this configuration by default. Second, MCTS agents become competitive only at simulation budgets exceeding 50,000 moves, making them less efficient than Alpha-Beta agents for this domain but potentially preferable in environments where constructing a reliable evaluation function is difficult. Third, the first-mover advantage findings suggest that tournament protocols for Mancala-based AI benchmarks should always use seat-swapping to avoid confounding agent strength with seating position. Neller's (2022) fair starting positions offer an alternative approach that removes the structural advantage entirely.

7.5 Limitations

Matchup sample sizes were limited to 50 games, producing wide confidence intervals. Logistic extrapolation for the MCTS agents' scaling assumes monotonic behavior when actual performance could plateau if random rollouts create a quality ceiling. Instability in Elo ratings affects agents with few opponents, which is a trend that is reflected in their wider bootstrap confidence intervals. Reliance on a single game variant of Mancala means that results may not generalize to other Mancala variants with different stone counts or board sizes. Manual tuning of evaluation function weights, rather than algorithmic optimization, may limit the findings. Purely random rollouts constrained the MCTS agents, however heuristic-guided rollout policies (Rovaris, 2017; Browne et al., 2012) could substantially improve their performance and shift crossover points. Lastly, the logit-linear depth extrapolation assumes that each depth increment has a constant effect in logit space, an assumption that might fail if transposition redundancies reduce the effective branching factor at deeper search.

7.6 Future Work

Implementing heuristic-guided MCTS agent rollouts or the implicit Minimax backup method that Lanctot et al. (2014) presents would test whether hybrid approaches close the gap between MCTS agents and deep Alpha-Beta agent searches. Moreover, expanding the MCTS

agent simulation budgets to 200,000 and 500,000 simulations per move would validate the logistic predictions for MCTS agent scaling. Testing Alpha-Beta agents with a search depth of ten while using iterative deepening would clarify whether depth scaling extrapolations hold at higher levels. Applying Neller's (2022) fair starting positions could determine if the observed intelligence gradient persists once researchers remove the structural first-mover advantage. Finally, automated weight optimization for the evaluation function through genetic algorithms or Bayesian optimization would reveal whether the standard heuristic's advantage at a depth of four reflects genuine feature superiority or merely better-calibrated weights.

8 YOUR FEEDBACK

- The open-ended project format allowed the team to pursue a research question organically, moving from a planned 11-agent experiment to a 30-configuration tournament as the analysis revealed new dimensions worth investigating. This flexibility was the most valuable aspect of the project.
- The project also helped the team recognize the importance of clear documentation and reproducible analysis. Because the experiment expanded over time, maintaining organized code, consistent naming conventions, and well-documented statistical outputs became essential for interpreting results and preparing the final report.
- Future iterations of the course could benefit from a brief module on experimental design for game AI, particularly around controlling for positional advantages (such as first-mover bias) and normalizing computation time across hardware. These issues were not obvious at the outset and required significant mid-project troubleshooting.

9 REFERENCES

Browne, C., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A survey of Monte Carlo Tree Search methods. *IEEE Transactions on Computational Intelligence and AI in Games, 4*(1), 1–43. https://doi.org/10.1109/TCIAIG.2012.2186810

Coulom, R. (2008). Whole-history rating: A Bayesian rating system for players of time-varying strength. In H. J. van den Herik, X. Xu, Z. Ma, & M. H. M. Winands (Eds.), *Computers and games (Lecture Notes in Computer Science, Vol. 5131, pp. 113–124).* Springer. https://doi.org/10.1007/978-3-540-87608-3_11

Gifford, C., Bley, J., Ajayi, D., & Thompson, Z. (2008). *Searching and game playing: An artificial intelligence approach to Mancala*. https://fiasco.ittc.ku.edu/publications/documents/Gifford_ITTC-FY2009-TR-03050-03.pdf

Elo, A. E. (1978). The rating of chessplayers, past and present. Arco.

Hunter, T. (2021). *The exploration and analysis of Mancala from an AI perspective*. https://doi.org/10.32597/honors/257/.

Kocsis, L., & Szepesvári, C. (2006). Bandit-based Monte-Carlo planning. In J. Fürnkranz, T. Scheffer, & M. Spiliopoulou (Eds.), *Machine learning: ECML 2006* (Lecture Notes in Computer Science, Vol. 4212, pp. 282–293). Springer. https://doi.org/10.1007/11871842_29

Irving, G., Donkers, J., & Uiterwijk, J. (2000). Solving Kalah. *ICGA Journal, 23*(3). https://doi.org/10.3233/ICG-2000-23303

Lanctot, M., Winands, M. H. M., Pepels, T., & Sturtevant, N. R. (2014). *Monte Carlo tree search with heuristic evaluations using implicit minimax backups*. https://doi.org/10.48550/arXiv.1406.0486

Milvus. (n.d.). *What is the role of heuristics in AI reasoning?* Milvus. Retrieved March 10, 2026, from https://milvus.io/ai-quick-reference/what-is-the-role-of-heuristics-in-ai-reasoning

Mucci, T. (2024, October 21). *The History of Artificial Intelligence*. IBM. Retrieved March 5, 2026, from https://www.ibm.com/think/topics/history-of-artificial-intelligence

Neller, T. W. (2022). AI education matters: FairKalah model AI assignment. *AI Matters, 8*(1), 9–11. https://doi.org/10.1145/3544897.3544900

Ramanujan, R., & Selman, B. (2011). Trade-offs in sampling-based adversarial planning. *Proceedings of the International Conference on Automated Planning and Scheduling, 21*, 202–209. https://doi.org/10.1609/icaps.v21i1.13472

Robinson, G. (2025, April 29). *The Crucible of Intelligence: A History of AI Playing Games and Its Enduring Significance*. Medium. https://gregrobison.medium.com/the-crucible-of-intelligence-a-history-of-ai-playing-games-and-its-enduring-significance-f2e767c4a7b7

Rovaris, G. (2017). *Design of artificial intelligence for Mancala games*. https://www.politesi.polimi.it/handle/10589/134455

Savannah African Art Museum. (n.d.). *Workshops*. Savannah African Art Museum. Retrieved March 10, 2026, from https://www.savannahafricanartmuseum.org/2020-workshops/05-2

APPENDIX

Detailed Statistical Tables, Figures, and Supporting Analysis

Group 8: Shuyang Huang, Elliott Armstrong, Matthew Kolker
AI 801, Spring 2026

This appendix presents the statistical tables, figures, and methodology details supporting the analysis in the main report. All data derive from 9,408 deduplicated games across 117 matchup configurations and 30 agent configurations. Formula-driven analysis (win rates, confidence intervals, matchup counts) is in the companion Excel workbook. Python-computed analysis (Elo ratings, hypothesis tests, regression, curve fitting) is in the companion Jupyter notebook.

Appendix A: Complete Agent Summary and Elo Ratings

Table A-1 presents all agent configurations ranked by Elo rating. Elo is a pairwise comparison system originally developed for chess by Arpad Elo (1978) and later extended to multi-player and AI contexts by Coulom (2008). The system provides a single scalar summary of relative strength that accounts for opponent quality, unlike raw win rate which is confounded by the distribution of opponents each agent faced.

Elo Computation Methodology

Each agent begins at an initial rating of 1500. After each game, ratings are updated using the formula:

$$R' = R + K \times (S - E) \quad \textbf{(A1)}$$

where K = 32 is the update magnitude (controlling sensitivity to individual results), S is the actual score (1 for a win, 0.5 for a draw, 0 for a loss), and E is the expected score computed from the logistic function:

$$E = \frac{1}{1+10^{(R_{opponent}-R_{self})/400}} \quad \textbf{(A2)}$$

The expected score E reflects the probability that an agent with the current rating will win against an opponent with the given rating. When an agent wins a game it was expected to win, the rating change is small. When an agent wins an upset, the gain is large. The 400-point scale constant means that a 400-point rating difference corresponds to a roughly 10:1 expected win ratio.

Because Elo ratings are path-dependent (the final rating depends on the order in which games are processed), the full set of 9,408 games was shuffled into a random order and processed 20 times. The reported rating for each agent is the mean across all 20 runs. The 95% confidence interval is computed from the 2.5th and 97.5th percentiles of the per-run rating distribution.

Sensitivity Analysis

To verify that the K parameter does not materially affect the ranking, Elo was computed at K = 16, K = 32, and K = 64. Spearman rank correlations between K = 32 and each alternative were: K=16: ρ=0.9880; K=32: ρ=0.9933; K=64: ρ=0.9911. The near-perfect correlations ($\rho > 0.98$) confirm that agent rankings are robust to K selection. Lower K values produce more conservative ratings (smaller updates per game, tighter clustering), while higher K values produce more volatile ratings, but the rank order is preserved.

Table A-1

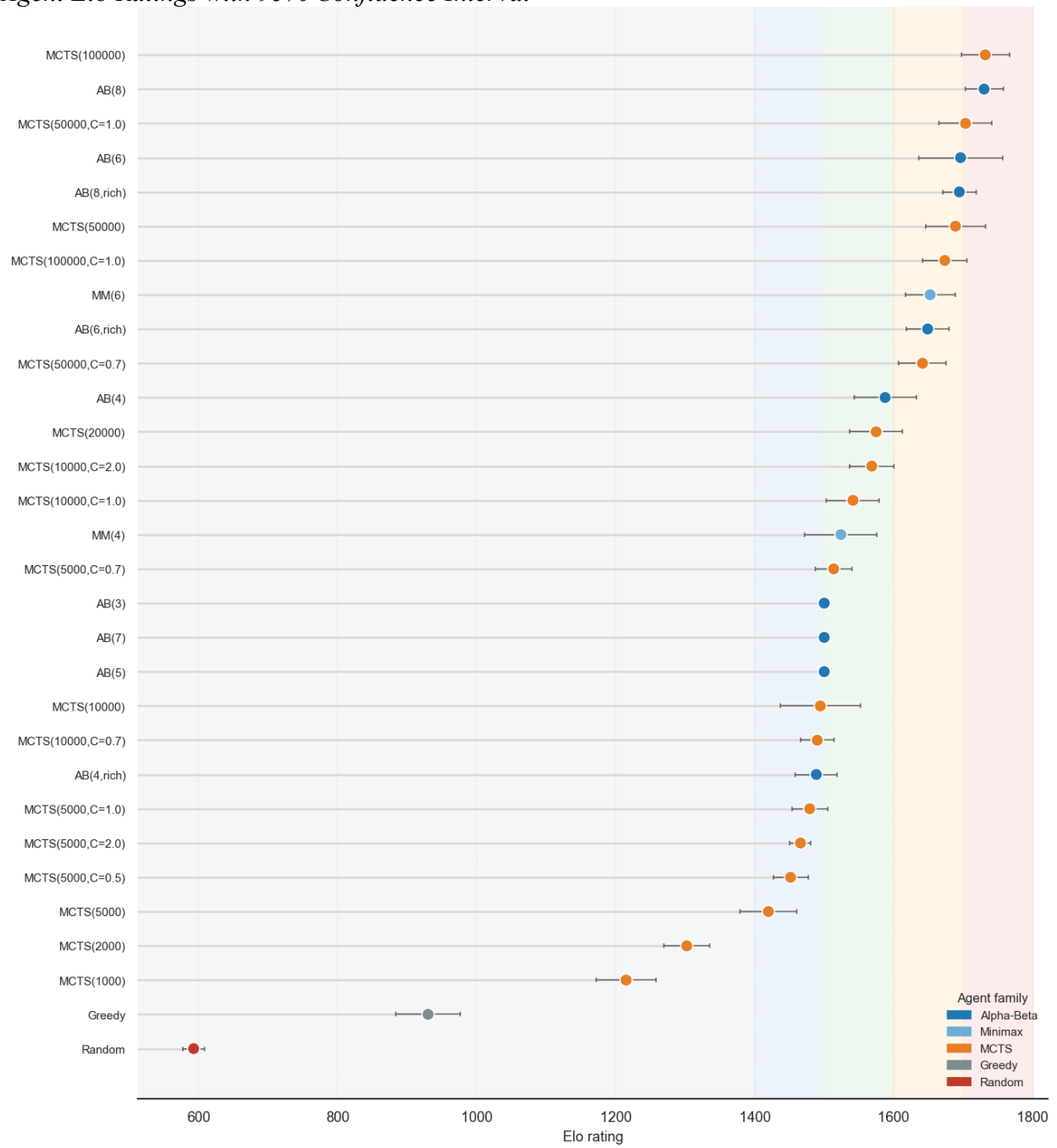
Agent Summary Ranked by Elo

#	Agent	Elo	95% CI	Games	W	L	D	WR	Chess	Score	Margin	ms	Tier
1	MCTS(n=100000)	1724	[1647,1784]	996	600	319	77	60%	0.641	25.1	+2.2	12.237	S
2	Alpha-Beta(d=8)	1716	[1643,1761]	1550	928	412	210	60%	0.666	27.5	+6.9	0.563	S
3	MCTS(n=50000,C=1.0)	1701	[1641,1748]	100	59	40	1	59%	0.595	23.2	-1.5	2.573	S
4	Alpha-Beta(d=8,rich)	1694	[1655,1731]	50	25	25	0	50%	0.500	24.9	+1.8	1.599	A
5	Alpha-Beta(d=6)	1694	[1595,1770]	1800	1103	622	75	61%	0.634	27.1	+6.3	0.065	A
6	MCTS(n=100000,C=1.0)	1680	[1630,1775]	100	58	42	0	58%	0.580	24.2	+0.5	11.457	A
7	MCTS(n=50000)	1669	[1600,1750]	1350	855	425	70	63%	0.659	25.5	+3.0	5.940	A
8	Alpha-Beta(d=6,rich)	1657	[1615,1704]	50	25	25	0	50%	0.500	28.3	+8.5	0.208	A
9	Minimax(d=6)	1656	[1576,1739]	850	566	258	26	67%	0.681	28.1	+8.1	1.076	A
10	MCTS(n=50000,C=0.7)	1636	[1580,1694]	100	50	48	2	50%	0.510	23.2	-1.5	2.537	A
11	Alpha-Beta(d=4)	1574	[1496,1641]	2200	1062	1088	50	48%	0.494	25.0	+2.0	0.006	B
12	MCTS(n=20000)	1569	[1506,1649]	1500	757	649	94	50%	0.536	23.7	-0.6	2.103	B
13	MCTS(n=10000,C=2.0)	1553	[1500,1626]	50	25	24	1	50%	0.510	22.4	-3.2	0.857	B
14	Minimax(d=4)	1545	[1458,1640]	950	503	386	61	53%	0.562	25.5	+3.0	0.019	B
15	MCTS(n=10000,C=1.0)	1539	[1482,1594]	50	24	25	1	48%	0.490	22.4	-3.2	0.819	B
16	MCTS(n=10000)	1510	[1448,1625]	1712	775	847	90	45%	0.479	23.0	-1.9	0.966	B
17	MCTS(n=5000,C=0.7)	1509	[1456,1557]	50	20	28	2	40%	0.420	21.9	-4.2	0.401	B
18	Alpha-Beta(d=7)	1500	[1500,1500]	100	50	50	0	50%	0.500	24.0	+0.0	0.305	B
19	Alpha-Beta(d=5)	1500	[1500,1500]	100	50	50	0	50%	0.500	24.0	+0.0	0.007	B
20	Alpha-Beta(d=3)	1500	[1500,1500]	100	50	50	0	50%	0.500	24.0	+0.0	0.001	B
21	Alpha-Beta(d=4,rich)	1494	[1446,1565]	100	27	56	17	27%	0.355	21.2	-5.6	0.028	C
22	MCTS(n=10000,C=0.7)	1493	[1451,1542]	50	20	29	1	40%	0.410	20.5	-7.0	1.141	C
23	MCTS(n=5000,C=1.0)	1489	[1451,1534]	50	20	29	1	40%	0.410	21.4	-5.2	0.405	C
24	MCTS(n=5000,C=2.0)	1470	[1426,1507]	50	18	31	1	36%	0.370	19.4	-9.1	0.748	C
25	MCTS(n=5000,C=0.5)	1453	[1406,1495]	50	16	32	2	32%	0.340	19.4	-9.1	0.396	C
26	MCTS(n=5000)	1420	[1380,1460]	1350	543	715	92	40%	0.436	22.5	-3.0	0.537	C
27	MCTS(n=2000)	1302	[1270,1334]	758	194	525	39	26%	0.282	20.5	-6.9	0.313	D
28	MCTS(n=1000)	1215	[1173,1257]	1150	311	774	65	27%	0.299	20.7	-6.5	0.138	D
29	Greedy	930	[884,976]	1050	164	850	36	16%	0.173	19.9	-8.2	0.000	D
30	Random	593	[578,608]	450	3	447	0	0.7%	0.007	12.1	-23.8	0.000	D

Tiers: S (≥ 1700), A (1600–1699), B (1500–1599), C (1400–1499), D (< 1400).

Figure A-1

Agent Elo Ratings with 95% Confidence Interval



Agents with only self-play data (A-B d=3, d=5, d=7) retain the default 1500 because no cross-agent matchups inform the update.

Appendix B: MCTS Scaling Analysis

Tables B-1 and B-2 present MCTS win rates and sample sizes for all tested opponents. Figure B-1 visualizes the scaling relationship against the three Alpha-Beta opponents with 95% Wilson confidence interval bands.

Table B-1

MCTS Win Rate by Simulation Budget and Opponent.

Sims	Greedy	MM(d=4)	MM(d=6)	AB(d=4)	AB(d=6)	AB(d=8)
1000	88%	17%	4%	26%	6%	0%
2000	100%	30%	10%	18%	12%	10%
5000	88%	30%	18%	49%	10%	2%
10000	96%	38%	29%	49%	22%	14%
20000	91%	40%	38%	46%	28%	31%
50000	92%	55%	51%	58%	62%	37%
100000	98%	60%	52%	58%	50%	47%

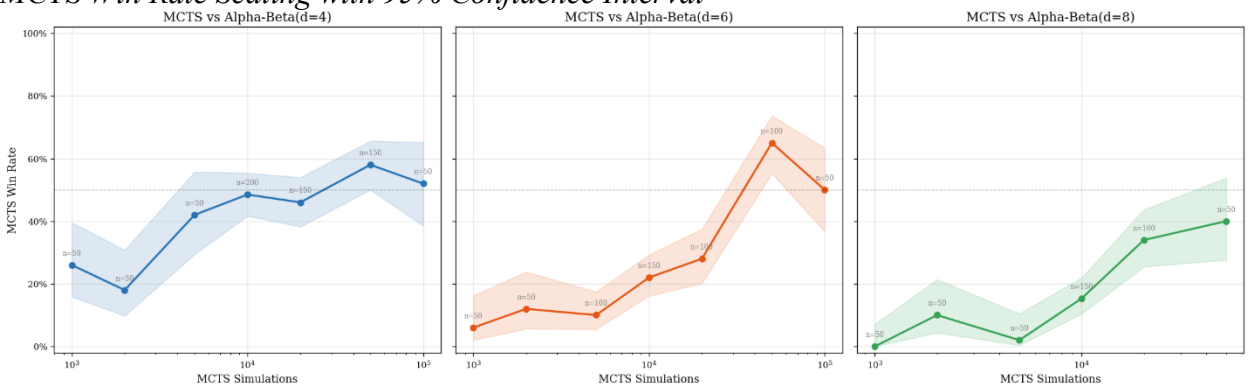
Table B-2

Games Played per MCTS Matchup.

Sims	Greedy	MM(d=4)	MM(d=6)	AB(d=4)	AB(d=6)	AB(d=8)
1000	100	100	50	50	50	50
2000	50	50	50	50	50	50
5000	100	100	100	100	100	50
10000	100	100	100	150	200	200
20000	100	100	100	200	150	150
50000	50	100	100	200	150	150
100000	50	50	50	200	200	150

Figure B-1

MCTS Win Rate Scaling with 95% Confidence Interval



Three distinct scaling regimes are visible. Against Greedy, MCTS saturates quickly (88% at 1,000 simulations, 98% at 100,000). Against Alpha-Beta(d=4), MCTS plateaus around 46% to 58% across 10,000 to 100,000 simulations, suggesting that random rollouts create a soft ceiling against moderate-depth adversarial search. Against Alpha-Beta(d=6) and Alpha-Beta(d=8), MCTS shows a steeper trajectory that has not yet plateaued, indicating that further budget increases would continue to improve performance.

Appendix C: Exploration Constant Sensitivity

The exploration constant C governs Equation (3). Higher C values encourage exploration of less-visited nodes; lower values favor exploitation of nodes with high observed rewards.

Table C-1

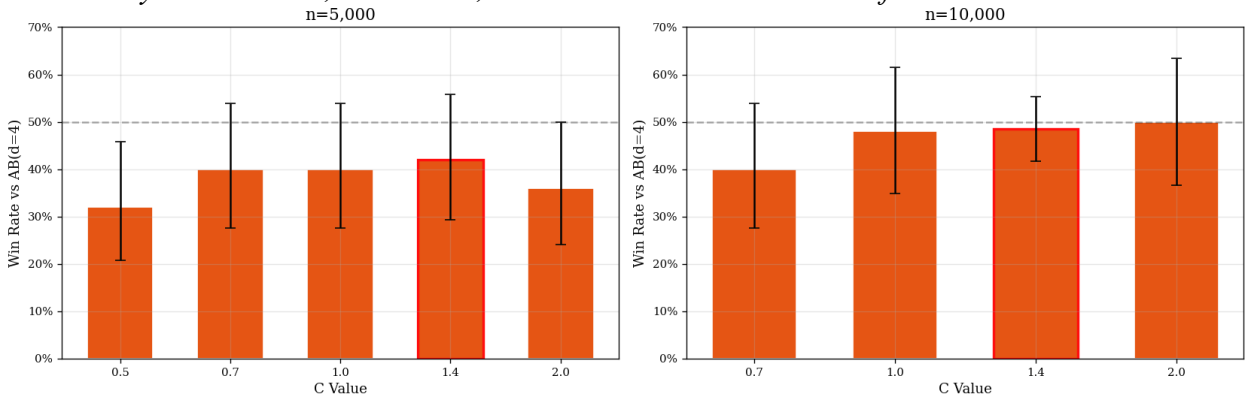
Exploration Constant Sensitivity vs. Alpha-Beta(d=4)

Config	Win Rate	95% CI	Games	Cohen's h
5k,C=0.5	32%	[21%,46%]	50	-0.368
5k,C=0.7	40%	[28%,54%]	50	-0.201
5k,C=1.0	40%	[28%,54%]	50	-0.201
5k,C=1.4	49%	[39%,59%]	100	-0.020
5k,C=2.0	36%	[24%,50%]	50	-0.284
10k,C=0.7	40%	[28%,54%]	50	-0.201
10k,C=1.0	48%	[35%,61%]	50	-0.040
10k,C=1.4	49%	[41%,57%]	150	-0.027
10k,C=2.0	50%	[37%,63%]	50	0.000

Cohen's h measures effect size relative to 50%

Figure C-1

Win Rate by C Value at 5,000 and 10,000 Simulations with 95% Confidence Interval



All confidence intervals overlap substantially, and all effect sizes are small ($|h| < 0.4$). The theoretical default C = 1.4, derived from $\sqrt{2}$ in Kocsis and Szepesvári (2006), is competitive across both budget levels. At 50-game sample sizes, the statistical power is insufficient to detect differences smaller than approximately 15 percentage points, so the overlapping CIs do not rule out small C effects that might emerge at larger sample sizes.

Appendix D: Evaluation Function Comparison

The standard evaluation function computes a normalized store differential: (P1_store − P2_store) / total_stones, returning a value between −1 and +1. The rich function uses single-move lookahead to estimate actual store gains and extra-turn counts for each legal move, combining four components with weights w1 = 0.50 (store differential), w2 = 0.20 (average store gain), w3 = 0.20 (extra-turn fraction), and w4 = 0.10 (stones-on-side differential).

Table D-1

Standard vs. Rich Evaluation Function

Matchup	Games	Std W	Rich W	Draws	Std WR	p-value	h
d=4	50	28	16	6	56%	0.480	0.120
d=6	50	25	25	0	50%	1.000	0.000
d=8	50	25	25	0	50%	1.000	0.000
Knowl. vs Search	50	11	28	11	22%	0.000	-0.594

At depth 4, the standard function won 28 of 50 games, suggesting the simpler heuristic retains a slight edge at shallow search, possibly because the single-move lookahead features in the rich function add computational noise without sufficient depth to act on the richer positional assessment. At depths 6 and 8, both functions are tied at 25-25, indicating convergence. The knowledge-versus-search comparison (row 4) yielded p = 0.009, confirming that two additional search plies outweigh a richer evaluation function, consistent with Ramanujan and Selman (2011).

Appendix E: Extra Turn Analysis

Extra turns are the core tactical mechanism in Mancala. When the last sown stone lands in a player's store, that player moves again immediately, enabling consecutive actions without opponent intervention. This appendix provides a comprehensive statistical analysis of extra turns and their relationship to game outcomes.

Table E-1

Extra Turn Descriptive Statistics (8,901 decided games)

Statistic	Winner	Loser
Mean	9.03	3.64
SD	5.30	3.04
Median	8	3
N	8,901	8,901

The Mann-Whitney U test ($U = 64{,}197{,}744$, $p < 10^{-300}$) and Welch's t-test ($t = 83.15$) both confirm the difference is highly significant. Cohen's $d = 1.246$ indicates a large effect. Winners earn roughly 2.5 times more extra turns than losers.

Table E-2

Extra Turns by Algorithm Family (Winners Only)

Algorithm	Mean ET	SD	N	Median
Alpha-Beta	12.74	4.41	3320	13
Minimax	12.29	4.97	1069	13
MCTS	5.56	3.19	4345	5
Greedy	4.60	3.42	164	4

A one-way ANOVA confirmed significant differences across families ($F = 2433.3$, $p < 0.001$). Search-based agents (Alpha-Beta at 12.74, Minimax at 12.29) earn roughly 2.3 times more extra turns than sampling-based agents (MCTS at 5.56, Greedy at 4.60) when winning. This gap reflects the depth extension mechanism: because extra turns do not consume search depth, Alpha-Beta and Minimax can deliberately plan multi-step extra-turn chains, whereas MCTS discovers them only through random simulation.

Table E-3

Pearson Correlation Matrix (Player 1 Perspective, 9,408 games)

	Score	Extra Turns	Captures	Moves
Score	1.000	0.503	0.326	-0.176
Extra Turns	0.503	1.000	0.189	0.340
Captures	0.326	0.189	1.000	0.209
Moves	-0.176	0.340	0.209	1.000

Table E-4

Score ~ Extra Turns + Captures + Total Moves. $R^2 = 0.476$

Predictor	β	r	p
Intercept	25.42		
Extra Turns	1.408	0.503	< 0.001
Captures	1.882	0.326	< 0.001
Moves	-0.390	−0.176	< 0.001

The model explains 47.6% of score variance. Each additional extra turn is associated with a 1.41-point increase in final score, controlling for captures and game length. Captures have a larger per-unit coefficient (1.88) but are less frequent. Game length is negatively associated with score, reflecting that longer games involve more balanced competition.

Figure E-1

Winner vs. Loser Turns

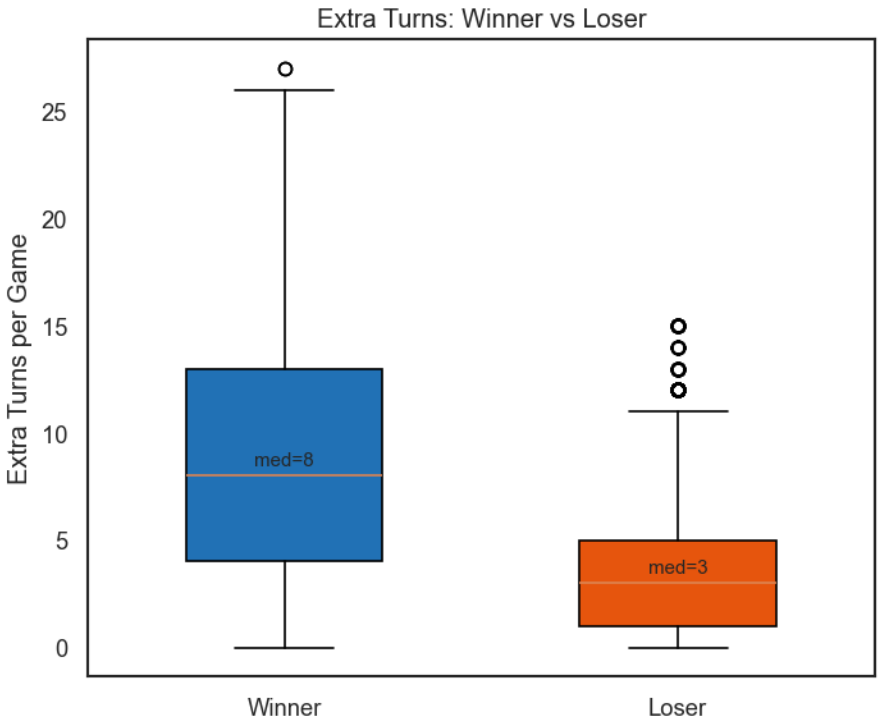


Figure E-2
Extra Turns vs. Score Scatter (r=0.503)

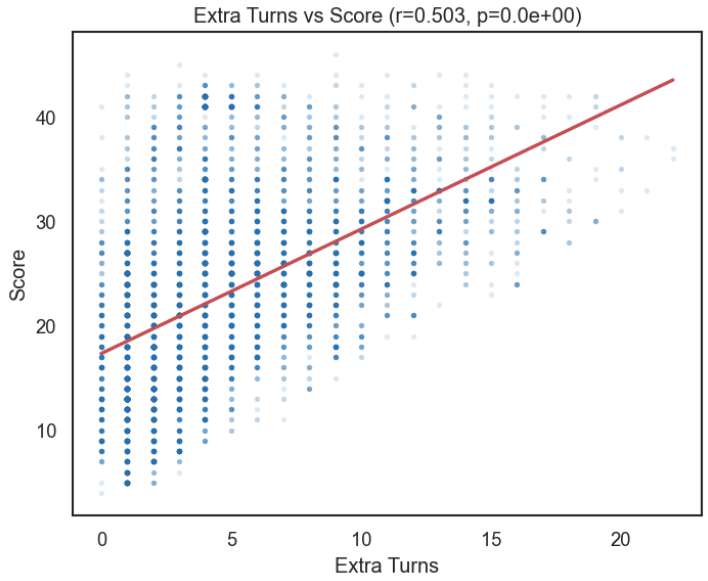
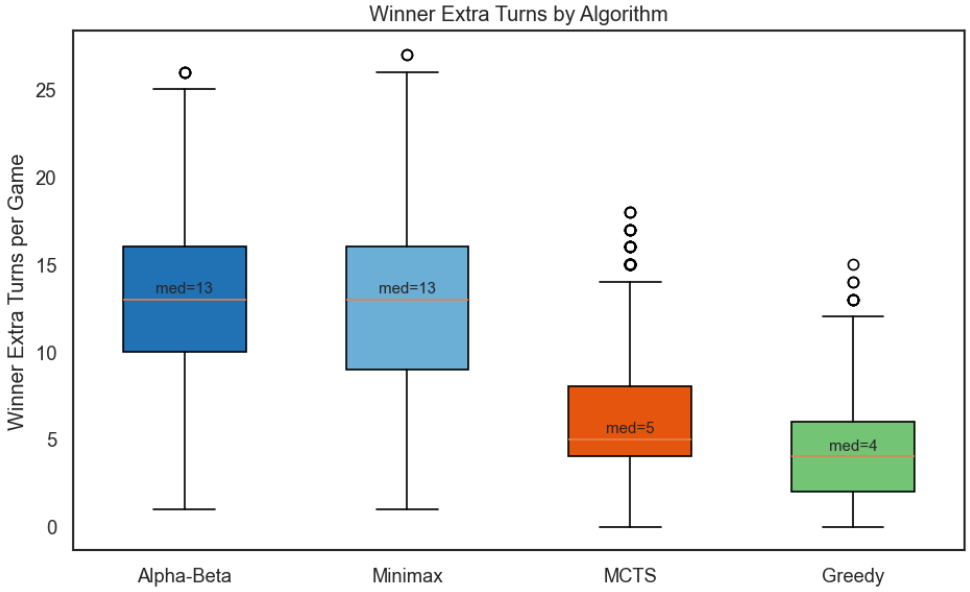


Figure E-3
Extra Turns by Algorithm Family when Winning



Appendix F: First-Mover Advantage

Table F-1

Self-Play First-Mover Results Sorted by Elo

Agent	Elo	Games	Decided	Draws	FM	SM	FM Rate	95% CI
MCTS(n=100000)	1724	50	47	3	36	11	77%	[63%,86%]
Alpha-Beta(d=8)	1716	100	56	44	56	0	100%	[94%,100%]
Alpha-Beta(d=6)	1694	100	100	0	100	0	100%	[96%,100%]
MCTS(n=50000)	1669	50	43	7	29	14	67%	[53%,80%]
Alpha-Beta(d=4)	1574	100	100	0	100	0	100%	[96%,100%]
MCTS(n=20000)	1569	100	96	4	80	16	83%	[75%,89%]
MCTS(n=10000)	1510	150	142	8	114	28	80%	[73%,86%]
Alpha-Beta(d=3)	1500	50	50	0	50	0	100%	[93%,100%]
Alpha-Beta(d=5)	1500	50	50	0	45	5	90%	[79%,96%]
Alpha-Beta(d=7)	1500	50	50	0	50	0	100%	[93%,100%]
MCTS(n=5000)	1410	100	92	8	63	29	68%	[58%,77%]
MCTS(n=1000)	1220	100	90	10	49	41	54%	[44%,64%]
Greedy	928	100	92	8	62	30	67%	[57%,76%]

Table F-2

First-Mover Advantage by Context

Context	Decided	FM Wins	FM Rate	95% CI	p-value
All decided	8901	6615	74.3%	[73.4%,75.2%]	0.0e+00
Self-play	1008	834	82.7%	[80.3%,84.9%]	7.3e-104
Cross-algorithm	7893	5781	73.2%	[72.3%,74.2%]	0.0e+00
|Margin| > 10	5099	4229	82.9%	[81.9%,83.9%]	0.0e+00
|Margin| <= 5	1697	1088	64.1%	[61.8%,66.4%]	1.6e-31
A-B self-play	406	401	98.8%	[97.1%,99.5%]	1.1e-111
MCTS self-play	510	371	72.7%	[68.7%,76.4%]	2.1e-25

Table F-3

Alpha-Beta Self-Play by Depth

Depth	Games	FM	SM	Draws	Draw%	FM%	Dec. Moves	Draw Moves
3	50	50	0	0	0%	100%	41.4	—
4	100	100	0	0	0%	100%	32.1	—
5	50	45	5	0	0%	90%	37.8	—
6	100	100	0	0	0%	100%	45.4	—
7	50	50	0	0	0%	100%	37.0	—
8	100	56	0	44	44%	100%	42.8	27.9

The draw rate jumped from 0% at depths 3 through 7 to 44% at depth 8. All 44 draws ended with the exact score 24-24, a perfect split of the 48 stones. Drawn games were substantially shorter (27.9 moves) than decided games (42.8 moves), suggesting that both agents detect the forced-draw path early. This threshold effect likely represents partial convergence toward the theoretical equilibrium identified by Irving et al. (2000).

Statistical Tests

A chi-square test for homogeneity of first-mover rates across self-play agents yielded $\chi^2 = 163.7$, df = 12, p = 9.05e-29, confirming that first-mover rates vary significantly by agent intelligence. The Spearman rank correlation between Elo and first-mover rate (stochastic agents only) was $\rho = 0.571$, p = 0.180.

Figure F-1

First-Mover Advantage Rate Sorted by Elo Rating

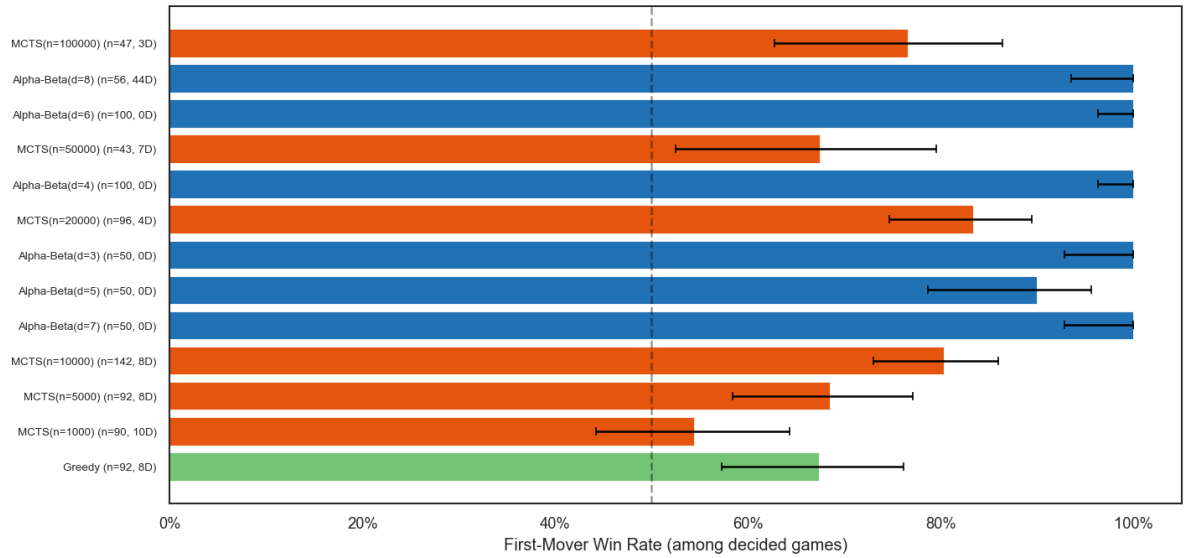
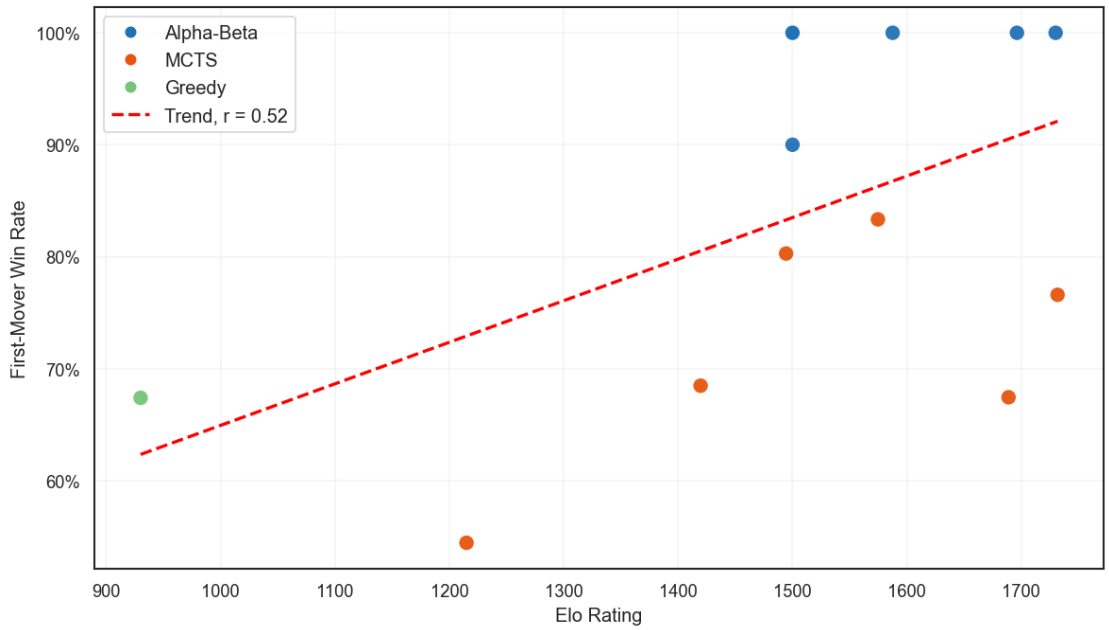


Figure F-2

First-Mover Advantage Rate vs. Elo Rating



Appendix G: Pruning Efficiency

Table G-1

Minimax vs. Alpha-Beta Timing and Head-to-Head

Depth	MM ms	A-B ms	Speedup	Games	MM W	A-B W	p	h
4	0.019	0.006	3.2x	50	27	22	0.672	0.080
6	1.076	0.065	16.4x	50	24	25	0.888	-0.040

Both effect sizes are negligible ($|h| < 0.1$), confirming identical competitive strength at matched depths. The 15.7x speedup at depth 6 means Alpha-Beta evaluates approximately 6% of the nodes that Minimax evaluates. The improvement comes from move ordering (extra-turn priority, store-gain sort) that increases the likelihood of early cutoffs.

Appendix H: Depth Scaling

Table H-1

Search Agent Win Rates Against Baselines

Agent	Opponent	Win Rate	Games	ms/move
Minimax(d=4)	Random	100%	50	0.019
Minimax(d=4)	Greedy	94%	50	0.019
Minimax(d=6)	Random	100%	50	1.076
Minimax(d=6)	Greedy	100%	50	1.076
AB(d=4)	Random	100%	50	0.006
AB(d=4)	Greedy	100%	50	0.006
AB(d=6)	Random	100%	50	0.065
AB(d=6)	Greedy	100%	50	0.065
AB(d=8)	Random	100%	50	0.563
AB(d=8)	Greedy	100%	50	0.563

Table H-2

Alpha-Beta Pairwise Win Rates.

	AB(d=4)	AB(d=6)	AB(d=8)
AB(d=4)	—	26% (150g)	33% (150g)
AB(d=6)	74% (150g)	—	50% (100g)
AB(d=8)	58% (150g)	50% (100g)	—

AB(d=6) vs AB(d=8): exactly 50-50 across 100 games

Appendix I: Cost-Performance Analysis

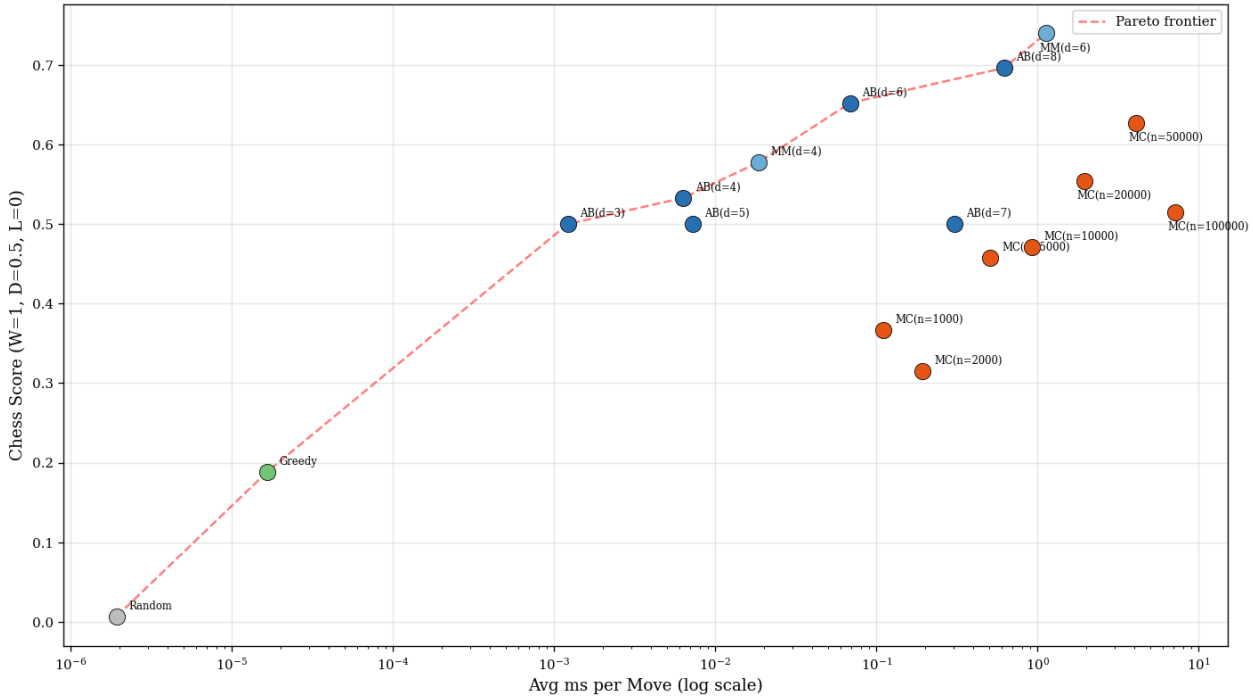
Table I-1

Cost-Performance for 15 Core Agents Excluding C-variants and Rich-Eval Variants

Agent	Elo	Chess Score	ms/move
MCTS(n=100000)	1724	0.641	12.237
Alpha-Beta(d=8)	1716	0.666	0.563
Alpha-Beta(d=6)	1694	0.634	0.065
MCTS(n=50000)	1669	0.659	5.940
Minimax(d=6)	1656	0.681	1.076
Alpha-Beta(d=4)	1574	0.494	0.006
MCTS(n=20000)	1569	0.536	2.103
Minimax(d=4)	1545	0.562	0.019
MCTS(n=10000)	1510	0.479	0.966
Alpha-Beta(d=7)	1500	0.500	0.305
Alpha-Beta(d=5)	1500	0.500	0.007
Alpha-Beta(d=3)	1500	0.500	0.001
MCTS(n=5000)	1410	0.436	0.537
MCTS(n=2000)	1323	0.282	0.313
MCTS(n=1000)	1220	0.299	0.138

Figure I-1

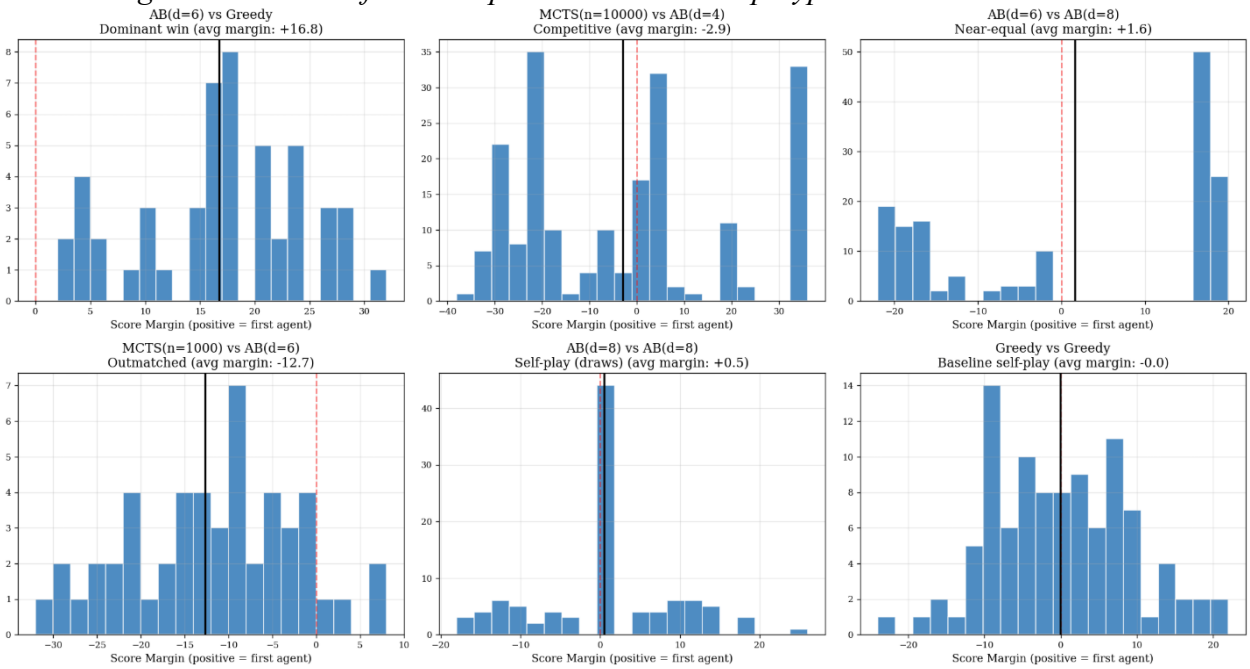
Cost-Performance Pareto Frontier.



AB(d=6) at 0.068 ms/move is Pareto-optimal for high-strength play

Figure I-2

Score Margin Distributions for Six Representative Matchup Types



Appendix J: Predictive Modeling

MCTS Logistic Scaling Model

Logistic regression models were fit to MCTS win rates as a function of $\log_{10}$(simulation budget), weighted by sample size. The model is:

$$\boldsymbol{Win\ Rate} = \frac{1}{1 + e^{-(a \times log_{10}(sims) + b)}} \quad \textbf{(J1)}$$

where a and b are fitted parameters. The delta method was used to propagate parameter uncertainty into prediction confidence intervals.

Table J-1

Logistic Model Parameters and Predictions with 95% Confidence Intervals

Opponent	a (±SE)	b (±SE)	R^2	200k sims	500k sims	1M sims
Alpha-Beta(d=4)	0.666±0.189	-2.888±0.826	0.740	66% [56%,75%]	71% [60%,82%]	75% [63%,87%]
Alpha-Beta(d=6)	1.547±0.404	-7.429±1.840	0.863	68% [53%,83%]	80% [64%,96%]	86% [72%,100%]
Alpha-Beta(d=8)	1.579±0.252	-7.936±1.166	0.920	61% [51%,70%]	74% [63%,85%]	82% [72%,93%]

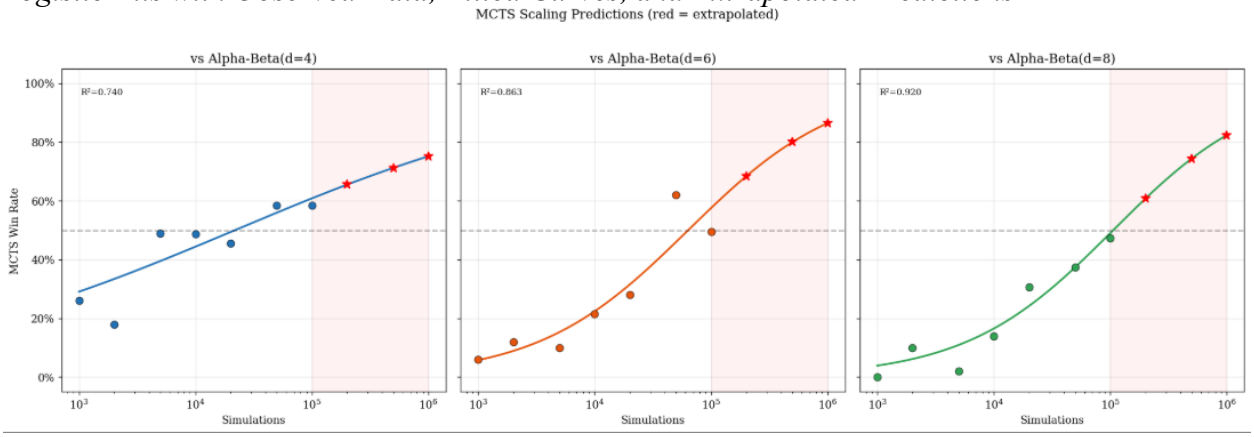
Table J-2

Observed and Predicted MCTS Win Rate by A-B Depth

MCTS Budget	d=4	d=6	d=8	d=10 (pred)	d=12 (pred)
MCTS(n=10000)	49%	22%	14%	6%	2%
MCTS(n=20000)	46%	28%	31%	22%	17%
MCTS(n=50000)	58%	62%	37%	32%	23%
MCTS(n=100000)	58%	50%	47%	41%	35%

Figure J-1

Logistic Fits with Observed Data, Fitted Curves, and Extrapolated Predictions

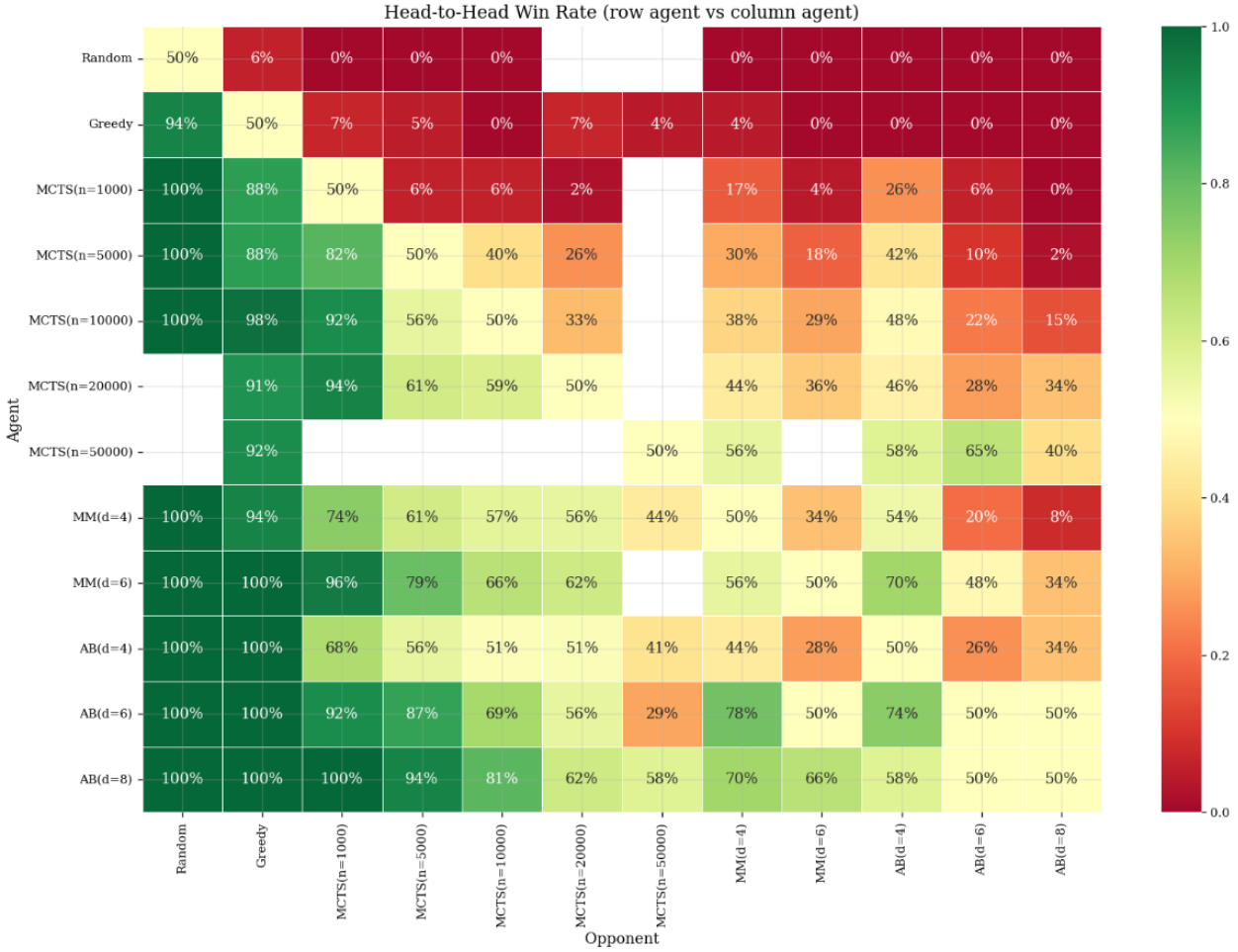


These predictions assume that the logistic relationship observed within the tested range continues monotonically. Several factors could invalidate this assumption: MCTS win rates may plateau if random rollouts create a quality ceiling, Alpha-Beta at higher depths may encounter diminishing returns from transposition redundancies, and the 50-game samples at the extremes produce wide CIs that propagate into the predictions.

Appendix K: Head-to-Head Win Rate Matrix

Figure K-1

Pairwise Win Rate Heatmap



Agent \ Opponent	Random	Greedy	MCTS(n=1000)	MCTS(n=5000)	MCTS(n=10000)	MCTS(n=20000)	MCTS(n=50000)	MM(d=4)	MM(d=6)	AB(d=4)	AB(d=6)	AB(d=8)
Random	50%	6%	0%	0%	0%			0%	0%	0%	0%	0%
Greedy	94%	50%	7%	5%	0%	7%	4%	4%	0%	0%	0%	0%
MCTS(n=1000)	100%	88%	50%	6%	6%	2%		17%	4%	26%	6%	0%
MCTS(n=5000)	100%	88%	82%	50%	40%	26%		30%	18%	42%	10%	2%
MCTS(n=10000)	100%	98%	92%	56%	50%	33%		38%	29%	48%	22%	15%
MCTS(n=20000)		91%	94%	61%	59%	50%		44%	36%	46%	28%	34%
MCTS(n=50000)		92%					50%	56%		58%	65%	40%
MM(d=4)	100%	94%	74%	61%	57%	56%	44%	50%	34%	54%	20%	8%
MM(d=6)	100%	100%	96%	79%	66%	62%		56%	50%	70%	48%	34%
AB(d=4)	100%	100%	68%	56%	51%	51%	41%	44%	28%	50%	26%	34%
AB(d=6)	100%	100%	92%	87%	69%	56%	29%	78%	50%	74%	50%	50%
AB(d=8)	100%	100%	100%	94%	81%	62%	58%	70%	66%	58%	50%	50%

The matrix reveals clear hierarchical structure. Random loses to everything. Greedy loses to everything except Random. MCTS agents show progressive improvement with simulation budgets. The diagonal (self-play) is always 50% by definition. Off-diagonal asymmetry reflects competitive imbalance. The full numeric matrix is available in the Excel workbook on the Head-to-Head sheet, computed entirely with COUNTIFS formulas.

Appendix L: Anomalies and Unexpected Patterns

This section documents nine anomalous or unexpected patterns identified in the experimental data. Each is described, quantified, and classified by severity.

L.1 Alpha-Beta(d=5) Second-Mover Wins

At depth 5, the first mover won only 45 of 50 decided games (90%), while all other depths produced 100%. The five second-mover wins showed consistent patterns: the second mover earned equal or more extra turns in two cases, and three games had nearly identical profiles (25 to 23 score, 6 versus 5 extra turns, 28 moves), suggesting a reproducible opening path. The probable cause is an odd-depth interaction with the depth extension, which creates asymmetric search horizons at odd nominal depths. Severity: low.

L.2 MCTS Scaling Non-Monotonicity

MCTS(100,000 simulations) versus Alpha-Beta(d=6) dropped from 62% at 50,000 to 50% at 100,000, with only partial CI overlap. Machine-level analysis showed no hardware confound (49% on machine A, 50% on machine B). Four other non-monotonic instances had fully overlapping CIs. Severity: moderate for the 100,000-simulation case; low for all others.

L.3 MCTS Self-Play FM Drop at 50,000

The first-mover rate dropped from 83% at 20,000 simulations to 67% at 50,000 (only 43 decided games, CI [53%, 80%]). The higher draw rate at 50,000 (14% versus 4%) shifts some first-mover wins into draws. Severity: low.

L.4 MCTS Performs Better Against Stronger Opponent

MCTS(20,000) won 31% against Alpha-Beta(d=8) but only 28% against Alpha-Beta(d=6). CIs overlap entirely ([24%, 38%] versus [21%, 36%]). A secondary explanation is style-dependent matchup variance. Severity: low.

L.5 Alpha-Beta(d=8) Draw Discontinuity

Draws jumped from 0% (depths 3 through 7) to 44% at depth 8. All 44 draws ended exactly 24 to 24. Drawn games averaged 27.9 moves versus 42.8 for decided games. This is a search threshold effect, not an error: at depth 8, both agents detect forced-draw opening sequences (Irving et al., 2000). Severity: informative finding.

L.6 Minimax(d=4) vs Greedy: 94% Gap

Minimax(d=4) won 94% versus Alpha-Beta(d=4)'s 100%. Both losses came as second mover. Alpha-Beta's move ordering breaks ties in favor of extra-turn moves; Minimax breaks ties randomly. Severity: low.

L.7 Machine-Dependent FM Rates

Aggregate rates: 74.1% (machine A) versus 79.2% (machine B). Investigation revealed Simpson's paradox: different matchup portfolios per machine. Within 23 shared matchups, rates were indistinguishable. Severity: none (explained confound).

L.8 MCTS vs Greedy Sub-100% Ceiling

Win rates fluctuated 88% to 100% across budgets. True rate approximately 92% to 98%. Greedy occasionally exploits board states where myopia aligns with optimal strategy. Severity: low.

L.9 No Perfect 48-0 Games

Maximum score: 46 (two games). Mancala's capture mechanics create a floor on the losing player's score because captures require opponent stones in pits opposite empty pits. Game property, not anomaly. Severity: none.

Table L-1

Anomaly Summary

#	Anomaly	Severity	Cause
1	AB(d=5) SM wins	Low	Odd-depth extension
2	MCTS 100k regression	Moderate	Sample variance
3	MCTS FM at 50k	Low	Small sample
4	MCTS vs harder opp	Low	CI overlap
5	d=8 draw jump	Informative	Search threshold
6	MM(d=4) gap	Low	Move ordering
7	Machine FM rates	None	Simpson's paradox
8	MCTS vs Greedy	Low	Stochastic ceiling
9	No 48-0 games	None	Game mechanics